Pagewire: Workload-Adaptive Rewiring of Disk-Resident Graph Indexes

Minghao Li
University of Toronto
Toronto, Canada
mingh.li@mail.utoronto.ca

Nick Koudas
University of Toronto
Toronto, Canada
koudas@cs.toronto.edu

ABSTRACT

Disk-resident graph indexes support approximate nearest-neighbor search over vector collections that exceed main memory, but their graphs are typically built before the workload is known. When queries concentrate on a small portion of the collection, search may follow unnecessary paths and incur avoidable block reads. We present Pagewire, a background method that adapts both the graph edges and the physical block layout of a live DiskANN index using served-query results and access traces, supplemented by occasional higher-effort searches. Pagewire protects selected long-range build edges while learning workload-specific replacements, uses co-visitation to improve block locality, and employs cost-aware edge selection and a minimum-gain gate to control I/O and write overhead with limited recall loss. Background corpus-target audits monitor failures and adjust the protection level. Our evaluation against existing disk-resident adaptation methods shows that Pagewire improves recall by up to 16% and achieves up to 1.5× higher QPS than existing methods. We further show that Pagewire adapts as query distributions change and that its cost controller reduces adaptation cost while sacrificing little recall.

KEYWORDS

approximate nearest-neighbor search, graph indexes, workload adaptation, vector databases

PVLDB Reference Format:
Minghao Li and Nick Koudas. Pagewire: Workload-Adaptive Rewiring of Disk-Resident Graph Indexes. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/ming-afk/Pagewire.

1 INTRODUCTION

Approximate nearest-neighbor (ANN) search retrieves vectors close to a query vector and is widely used in retrieval and recommendation systems. At large scale, the vector collection and its index may exceed main memory. DiskANN [22] supports search in this setting by keeping compressed vectors in memory and a graph on disk. Search uses the compressed vectors to select nodes, reads their records, and re-ranks the examined vectors using full-precision distances. The graph determines the routes available to search, while the layout determines the blocks it must read. Both therefore affect the cost of answering a query, yet they are typically constructed before the query workload is known.

Consider a retrieval service over technical documentation. After a software release, queries may concentrate on installation and troubleshooting. If search repeatedly reaches a useful document through several intermediate nodes, an edge from a frequently visited node to that document can shorten the route. Similarly, if these queries visit two records in separate uncached blocks, placing the records together, when they fit, can reduce two reads to one. The workload thus provides evidence for improving both graph edges and record placement. Later, queries may shift to another feature, making different routes and placements useful even though the collection has not changed. The index must adapt to these changes while continuing to serve queries, including occasional requests outside the popular part of the collection.

Using the workload to improve physical organization has a long history in databases, including index selection [4] and database cracking [10]. Applying this principle to a disk-resident graph introduces several constraints. Each node has a limited number of outgoing edges, so adding a useful edge may require removing another. Frequent changes incur writes, and the statistics needed to choose these changes consume memory. Moreover, an edge that is useful for answering queries need not be inexpensive to access: its destination may require a new block read. Adaptation must therefore account for routing, access, and maintenance cost together.

We present Pagewire, a background method that adapts the edges and physical layout of a live DiskANN index. It retains selected long edges from the original build and learns replacements for the remaining edges within each node's degree budget. Background searches for sampled corpus targets identify failures and guide repairs and changes to the protection level. The learner accounts for I/O cost when ranking candidates and requires sufficient estimated gain before committing a change. Block reorganization uses observations of which records queries visit together. The serving search retains its beam size, ranking, and re-ranking steps.

Pagewire obtains feedback from the queries it serves. After a query expands a node, the learner checks which candidate neighbors occur in the returned result, including candidates that are not current edges. This supplies observations without an additional traversal solely to obtain a label; Section 9 examines the effect of label quality. Learning state must fit within a fixed memory budget. Keeping just 1 KB for every node would require approximately 1 TB for a billion-node index. We therefore maintain this state only for a bounded set of frequently visited nodes, where queries supply the most observations.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Our contributions are as follows.

- We develop a length-based protection rule that retains selected build edges while the learner changes the remaining outgoing edges. Protection is needed because edges with little observed benefit can still provide routes to other parts of the collection. The analysis characterizes the progress preserved by these edges under stated assumptions, while corpus-target audits monitor failures during adaptation (Sections 4 and 5).
- We introduce a cost-adjusted score and a feedback-controlled weight that connect edge selection to an operator-specified I/O target. The score distinguishes similarly useful candidates by the additional block reads they are expected to incur. A minimum-gain gate limits commit frequency, with write bounds that account for score variation and protection-level changes (Section 6).
- We bound learner memory with a cache of frequently visited nodes. The analysis relates its size to the workload's visitation distribution and the observations needed to distinguish candidates (Section 7).
- We develop a block-reorganization method based on how often queries visit pairs of nodes together. For a fixed trace distribution, we relate this objective to expected I/O savings and identify an exact matching formulation when a block holds two nodes (Section 8).

2 BACKGROUND AND PROBLEM

2.1 Disk-resident graph indexes

Let $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$ be the vector collection, with distance $\mathrm{dist}(\cdot,\cdot)$. VAMANA [22] represents this collection as a directed graph $G = (X, E)$. Each node u has an outgoing list $N_{\mathrm{out}}(u)$ containing at most R neighbors. Construction selects these neighbors using RobustPrune(u, C, α, R). Here, u is the node being processed, C is its candidate-neighbor set, α controls the distance test used to prune candidates, and R is the maximum number of outgoing edges. The procedure adds the remaining candidate p^* closest to u to the out-list, then discards each other candidate p satisfying $\alpha\mathrm{dist}(p^*, p) \le \mathrm{dist}(u, p)$. Thus, a candidate is discarded when a retained neighbor is sufficiently closer to it than u is. This process continues until the list contains R neighbors or no candidates remain. During search, a beam holds at most L candidates and expands the W closest unexpanded candidates in each round.

DiskANN stores product-quantized vectors [13] in memory and full-precision vectors and adjacency lists on disk. Quantized distances guide search, and fetched vectors support the final re-ranking. Let $\pi : X \to \mathcal{B}$ map node records to blocks of capacity κ. Expanding node u requires its block $\pi(u)$. We measure logical I/O as

$$\mathrm{IO}(q) \;=\; \big|\pi(\mathcal{V}(q)) \setminus C_{\mathrm{blk}}\big|, \tag{1}$$

where $\mathcal{V}(q)$ contains the nodes expanded for query q and C_{blk} is the resident block cache. This expression counts each block absent from the cache once per query. It assumes that fetched blocks remain available for that query; measurements at the device also include any additional requests made by the implementation. We write $\bar{v} = \mathbb{E}_q|\mathcal{V}(q)|$ for the mean number of node expansions per query. This quantity is used to express learning loss per query (Section 7 and Appendix E) and to set the controller step size (Section 10).

To analyze progress toward a target, we use the following property, defined in terms of exact distances.

Definition 2.1 (α-navigability). A directed graph G on X is *α-navigable* ($\alpha > 1$) if for every node u and every target $y \in X$ with $u \neq y$ there exists $v \in N_{\mathrm{out}}(u)$ with $\mathrm{dist}(v, y) \le \mathrm{dist}(u, y)/\alpha$.

From any node other than the target, this property provides an edge that reduces the remaining distance by a factor of at least α. Repeating such steps reduces the distance by that factor at every hop. Appendix B gives the covering and hop bounds for exact distances. Section 4 explains how these results relate to quantized beam search.

2.2 The online problem

Queries arrive from an unknown distribution $\mathcal{D}$. Let G_0 denote the deployed graph. Query q_t is served on G_{t-1}, and G_t denotes the graph after that query. For a node u with positive visitation probability, we define the *true conditional utility* of candidate v at u as

$$Q^\star(u, v) \;\triangleq\; \mathbb{P}_{q\sim\mathcal{D}}\big[\, v \in N_k(q) \;\big|\; u \in \mathcal{V}(q) \,\big], \tag{2}$$

where $N_k(q)$ is the true top-k result. This utility measures how often v is a true answer to queries that expand u, regardless of whether search follows edge (u, v). The learner maintains a pool $\mathcal{P}(u)$ of at most C candidate-neighbor identifiers for u. For each candidate, $\widehat{Q}(u, v)$ estimates the probability that v appears in a query's answer labels, conditional on that query expanding u (Section 6.1). Exact labels yield estimates of $Q^\star$; approximate labels yield estimates of the utility defined by those labels.

Let $\mu_G(u) = \mathbb{P}_q[u \in \mathcal{V}_G(q)]$ denote the chance that a query expands u in graph G. Let V_+ contain the nodes with positive visitation and write $n_+ = |V_+|$. The reference point for learning is a value-greedy fixed point: each node retains its protected edges and fills its remaining entries with the highest-utility eligible candidates under the traffic generated by that graph, preserving its out-degree. We analyze these choices through the local objective

$$F(G) \;=\; \mathbb{E}_q\Big[\textstyle\sum_{u\in\mathcal{V}(q)} |N_{\mathrm{out}}(u) \cap N_k(q)|\Big]. \tag{3}$$

This objective counts, on average, true answers in the outgoing lists of expanded nodes. The same answer contributes again if it occurs in another expanded node's list. This local count is easier to optimize, but it differs from the recall of the returned result, as discussed in Appendix A.

The learning results use three assumptions, formalized in Appendix A. A1 assumes independent queries from a fixed, possibly highly concentrated workload, with a minimum positive chance of visiting each analyzed node throughout adaptation. A2 requires a positive gap in true utility between the last selected and first excluded unprotected candidates, ranked within the node's replacement quota. A3 limits the change in each candidate's conditional utility across graphs to less than half this gap. Together, these assumptions provide continuing observations and keep the set of best candidates under true utility unchanged as edges change. Retaining routing edges motivates A3 but does not guarantee it.

Before adaptation, the operator sets $I^\star$, the target mean block reads per query in (1). We adapt edges and placement to meet this target and a corpus-reachability target while limiting maintenance

Table 1: Notation.

Symbol	Meaning
R, $R_\ell(u)$	degree budget; replaceable entries at u; $R_\ell = \max_u R_\ell(u)$
S	protected set $P_{k_\mathrm{p}} \cup$ pin (§4)
$P_j(u)$, k_p, K_max	j longest build edges of u; level; record depth
$r_u(j)$	landing radius: length of u's j-th longest edge
$\mathrm{Reach}(G)$, ε_r	corpus reachability (5); target tolerance
p_a, p_u	audit probability per query; uniform-target probability per audit
κ, π	block capacity (nodes/sector), layout map
C, $\mathcal{P}(u)$	pool bound ($C \geq R$); candidate pool of u
B	rewire batch period (queries)
H, $\mathcal{H}$	hot-cache capacity (nodes); the cache itself
θ, λ	I/O cost weight; write-hysteresis threshold
$I^\star$, $\widehat{I}$	target mean serving I/O; measured mean serving I/O

Algorithm 1: PageWire online loop (commits every B queries).

Input: Vamana index G_0; level k_p (§4); audit rate p_a, uniform-target probability p_u (§5.1); params k, L, W, R, C, B, H; I/O target $I^\star$; write threshold λ; probe rate p, effort L_pr; step size $\beta_{u,v}$

Initialize: $\theta \leftarrow 0$; hot-state cache $\mathcal{H} \leftarrow \emptyset$; $\widehat{Q} \leftarrow$ prior k/n

for $t = 1, 2, \ldots$ **do**
- $\mathcal{V}_t \leftarrow \textsc{BeamSearch}(G_{t-1}, q_t, L, W)$ // unmodified DiskANN
- $\mathcal{H}.\textsc{Touch}$(unflagged nodes in $\mathcal{V}_t$) // admit/renew hot state (§7)
- **if** *Bernoulli*(p) **then**
  - $T_t \leftarrow \textsc{ProbeSearch}(q_t, L_\mathrm{pr} \gg L)$ // §9
  - $M_t \leftarrow T_t \setminus \mathrm{top}_k$(re-ranked examined set)
- **else**
  - $T_t \leftarrow \mathrm{top}_k$(re-ranked examined set); $M_t \leftarrow \emptyset$ // free self-label
- **end**
- **foreach** $u \in \mathcal{V}_t \cap \mathcal{H}$ **do**
  - **foreach** $v \in \mathcal{P}(u)$ *with state before this query* **do** $\widehat{Q}[u][v]$ += $\beta_{u,v}(\mathbf{1}[v \in T_t] - \widehat{Q}[u][v])$
  - update $\widehat{c}(u, \cdot)$ from recorded availability, and $\mathrm{co}(u, \cdot)$; $B_{nn}[u] \cup= T_t$;
  - **if** $M_t \neq \emptyset$ **then** $B_\mathrm{miss}[u] \cup= M_t$
- **end**
- **if** *Bernoulli*(p_a) **then** AuditStep() (Alg. 2)
- **if** $t \bmod B = 0$ **then**
  - $\textsc{CommitBatch}(\mathcal{H}, \theta, \lambda, I^\star)$ // Alg. 4
  - $\pi \leftarrow \textsc{LayoutStep}(\pi, \mathrm{co}, \lambda_\pi)$ // §8
  - $k_\mathrm{p} \leftarrow \textsc{Governor}(k_\mathrm{p})$ // §5
- **end**

end

writes and learner memory. Section 6.2 explains the controller; Table 1 summarizes notation.

3 SYSTEM OVERVIEW

Algorithm 1 shows how query processing supplies feedback for edge selection, block placement, and protection control. The *protection level* k_p determines how many of each node's longest build edges are retained. These edges support routes that frequent queries may not reward. The learner replaces the $R_\ell(u)$ unprotected entries at node u within the degree budget R. Section 4 defines the protected set, including pinned repair edges and nodes whose full build lists remain protected.

For each query q_t, search returns the expanded-node set $\mathcal{V}_t$ using the parameters in Section 2. The learner cache $\mathcal{H}$ holds statistics for at most H frequently visited nodes, bounding learning memory (Section 7). It is separate from the block cache, which holds records to avoid disk reads. Evicting learner statistics changes neither the graph nor the block cache.

The returned identifiers form the label set T_t. With probability p, a background *probe* repeats the query with a larger candidate-list limit L_pr and supplies its answers as labels instead. The probe may find answers missed by normal search; M_t records these additional answers. Section 9 describes the two label sources and their limitations.

At each visited node u with cached learner state, the learner updates $\widehat{Q}(u, v)$ for every candidate $v \in \mathcal{P}(u)$. This estimates how often v appears in the label set T_t on queries that visit u. The candidate-specific step size $\beta_{u,v}$ controls the weight of the new observation. Section 6.1 gives the estimator and explains how candidate pools are maintained. Two bounded buffers supply candidates: $B_{nn}[u]$ stores returned neighbors, while $B_\mathrm{miss}[u]$ stores missed answers and repair candidates. An answer found elsewhere in the graph can thus be proposed as a direct neighbor of u. The learner also updates $\widehat{c}(u, v)$, the estimated probability that the destination block is unavailable when u is expanded (Section 6.2), and $\mathrm{co}(u, v)$, which records how often two nodes are visited together (Section 8).

To check routes beyond those favored by the workload, AuditStep (Algorithm 2) searches for a sampled corpus target with probability p_a per served query. Failed audits recruit repair candidates (Section 5.1).

Every B queries, CommitBatch (Algorithm 4) adjusts $\theta \geq 0$, a weight that balances candidate utility against read cost. The learner scores each candidate by subtracting θ times its estimated read cost from its estimated utility. A larger θ therefore favors candidates less likely to require another block read. Measured mean reads per query above the target $I^\star$ raise θ; reads below the target lower it (Section 6.2). Since replacing edges incurs writes, a proposed set is committed only if its estimated cost-adjusted score improves by more than λ, the minimum gain for a learned-edge commit (Section 6.3).

LayoutStep then uses co-visitation to propose placing frequently co-visited records in the same block, potentially replacing separate reads with one. It updates the layout map π and applies a separate minimum-gain threshold λ_π to control relocation writes (Section 8). Finally, Governor (Algorithm 3) uses audit and edge-coverage statistics to adjust k_p (Section 5.2). Lowering the level releases a build-edge rank for possible replacement; raising it queues the required restorations. Released edges remain until a

learned commit replaces them, and a higher protection level takes effect after restoration.

4 TIERED PROTECTION

4.1 Why routing edges need protection

PAGEWIRE scores candidate neighbors by how often they appear in the answers to queries that visit a node. This captures their usefulness as answers, but an edge may also be useful as an intermediate step in search. It can lead to a region even when its destination rarely appears in query results.

Consider queries concentrated on one part of the collection. An edge leading toward a rarely queried region may receive a low score and be replaced. Search may continue to perform well in the popular region, concealing the loss of a route elsewhere. A later query for the neglected region may then miss relevant vectors. Since these vectors are absent from the returned answers, they receive no credit from that query, leaving ordinary feedback with little evidence for restoring the route.

PAGEWIRE addresses this risk by protecting selected edges from the original build and auditing search during serving. The learner can replace the remaining edges. Section 4.2 defines the protected set, and Section 4.3 explains the progress it preserves under the stated assumptions. Uniform corpus-target audits search for stored vectors independently of query popularity. Failures guide repairs and changes to the protection level (Section 5).

4.2 The length tier

At each node, PAGEWIRE protects a prefix of the longest build edges and leaves the remaining slots to the learner. Nodes outside the learner cache keep their existing out-lists. Length alone can leave an important route to a rarely queried region unprotected. PAGEWIRE can therefore *pin* a repair edge supported by repeated audit evidence, retaining it regardless of its length rank. Section 5.1 and Algorithm 2 explain how failed audits recruit and pin these edges.

Definition 4.1 (Length-tiered protection). For each node u, let $P_j(u)$ be the first $\min(j, |N^{(0)}_{\text{out}}(u)|)$ build edges in decreasing length order, with ties broken by identifier. At a fixed level k_{p}, let $S = P_{k_{\text{p}}} \cup \text{pin}$, where pin contains individually pinned edges. The protection invariant is

$$S(u) \subseteq N^{(t)}_{\text{out}}(u) \quad \text{for every } u. \tag{4}$$

For $1 \le j \le |N^{(0)}_{\text{out}}(u)|$, the landing radius $r_u(j)$ is the length of the j-th longest build edge. Set $r_u(0) = \infty$ and $r_u(j) = 0$ if $j > |N^{(0)}_{\text{out}}(u)|$. The active level k_{p} changes only after its required restorations are durable. A requested higher level remains pending until then. At flagged nodes, $S(u)$ also includes the full build out-list, which the method leaves unchanged.

Every replacement retains $S(u)$ and preserves the node's out-degree. Its replaceable quota is $R_\ell(u) = |N_{\text{out}}(u) \setminus S(u)|$, with $R_\ell(u) \le R - |S(u)|$; we use $R_\ell = \max_u R_\ell(u)$ in bounds. Flagged nodes have quota zero. A repair edge becomes pinned only after installation, and only if the node can also accommodate its full recorded prefix $P_{K_{\max}}(u)$ and all pins. This reserves space for future promotions. Protected entries precede the learned suffix. Appendix B.2 specifies level changes and pin installation; Appendix I gives the publication and recovery protocol.

4.3 Checking and maintaining protection

VAMANA constructs the initial graph using search and ROBUSTPRUNE, including pruning during reverse-edge processing [22, Section 2.3, Algorithm 3]. PAGEWIRE adds candidate logging and checks the completed graph. For each node, the log records the candidates considered during construction.

At node u, PAGEWIRE checks the prefix $P_j(u)$ of up to j longest build edges. For every logged target y beyond the landing radius $r_u(j)$ (Section 4.2), the prefix must contain a neighbor v satisfying

$$\text{dist}(v, y) \le \text{dist}(u, y)/\alpha.$$

Such an edge is a *progress witness*: following it reduces the distance to the target by a factor of at least α. A prefix that passes this check is *certified* at that level. PAGEWIRE checks every recorded level: changing the prefix also changes the targets beyond its landing radius, so certification at one level does not certify another. If the prefix at $K_{\max}$ fails, PAGEWIRE flags the node, retains its full build out-list, and disables learned rewiring there. These checks are PAGEWIRE additions to the VAMANA build.

Protection during serving. A *committed replacement* is an accepted change to unprotected outgoing edges that improves estimated cost-adjusted utility by more than λ and is queued for writing (Algorithm 4, Section 6.3). It retains the protected edges. When the active prefix is certified, this preserves its checked progress without repeating the coverage check.

Targets at or within the landing radius may depend on unprotected edges. For example, with landing radius 20 and $\alpha = 2$, a logged target at distance 12 needs a neighbor within distance 6 of that target. Replacing the only edge to such a neighbor removes this progress step. Other routes may remain, but removing the sole gateway to a region can make that region unreachable.

PAGEWIRE therefore monitors search during serving through background corpus-target audits. Uniformly sampled targets give rarely queried regions the same opportunity to be checked as popular regions. Failed audits supply repair candidates for the learner; repeated evidence can support pinning a repair edge. The governor can also increase protection and restore recorded build edges. Section 5 describes these decisions and the statistical scope of the audits.

Preservation result. The following statement summarizes what retaining a certified prefix establishes.

THEOREM 4.2 (PRESERVATION UNDER REWIRING; SUMMARY). *At a fixed protection level, retaining a certified prefix preserves α-progress toward every logged target beyond its landing radius, regardless of changes to the other edges. Under metric distances and the faithfulness assumption, this preservation extends to all corpus targets beyond that radius.*

Faithfulness links logged coverage to corpus coverage and, under the additional conditions in Appendix B, global α-navigability. Neither PAGEWIRE's checks and audits nor practical, degree-bounded VAMANA establish faithfulness. The ideal VAMANA guarantee uses full-corpus candidates without a restrictive degree cap [22, Sections 2.2–2.3]. PAGEWIRE preserves checked progress and uses audits to guide repairs. Appendix B.1 formalizes faithfulness, and

Theorem B.5 gives the navigability and hop bounds with proof in Appendix B.

5 AUDITS, REPAIRS, AND PROTECTION CONTROL

The degree budget R limits each node to R outgoing edges, one per adjacency-list entry, or *slot*. The protection level k_p retains up to k_p longest build edges at each node. Individually pinned edges also remain protected, and flagged nodes retain their full build lists (Section 4.2). The learner can replace only unprotected entries. The record depth K_{max} is the maximum number of longest build edges recorded per node for restoration; it also caps the protection level.

Good workload recall can conceal failures in rarely queried regions. The governor therefore searches for stored corpus targets. For the graph G being audited and corpus X, let $y \sim U(X)$ denote a target drawn uniformly from X. The audit success probability is

$$\text{Reach}(G) = \mathbb{P}_{y\sim U(X)}[\textsc{BeamSearch}(G, x_y, L_{aud}, W) \text{ examines } y \text{ or fetches } \pi(y)]. \quad (5)$$

Here x_y is the stored vector of target y, and $\pi(y)$ is its disk block. The audit candidate-list capacity is L_{aud} (L for ordinary queries); W is the number of closest unexpanded candidates expanded per search round (Section 2.1). $L_{aud} = 1$ (implemented as audit `ef=1`), so the audit tests greedy reachability without backtracking. An audit succeeds when search examines the sampled corpus target or fetches its block. This differs from workload recall, which measures how many true nearest neighbors appear in the returned answers. The operator specifies ε_r, the allowed uniform-target audit miss rate.

Build-time calibration sets the initial protection level $k_p^{(0)}$. The governor adjusts it during serving (Section 5.2). Appendix B.2 gives the calibration rule, record metadata, and restoration order.

5.1 Audits and repair

A background worker runs Algorithm 2 with probability p_a per served query. Lines 1–2 fix the audited state and draw a uniform corpus target with probability $p_u \in (0, 1]$, or a recent-insert target otherwise. Thus p_a controls frequency and p_u controls the mix. If there are no recent inserts, every target is uniform and is recorded as such.

Lines 3–5 search and record exactly one success or failure, tagged by sampling stratum. Only uniform-target outcomes enter the confidence bound. They give rarely queried and popular targets equal probability and can expose failures on unlogged targets or after rewiring, including cases where PQ search misses an exact-distance progress edge. The state is held fixed from before target selection until the outcome is recorded (see the publication protocol in Appendix I).

On failure, lines 6–7 select a few expanded nodes closest to target y and examine each for repair. Line 8 checks its *reserve edges*: recorded build edges at length ranks $k_p + 1$ through K_{max}. Rank 1 is the longest build edge; stored identifiers let the worker propose a removed edge again. The test seeks a neighbor v satisfying $\text{dist}_{PQ}(v, x_y) \le \text{dist}(u, x_y)/\alpha - m_{pq}$. Here $\alpha > 1$ is the progress factor, $\text{dist}(u, x_y)$ uses full precision, and $m_{pq} \ge 0$ is a margin for PQ underestimation, in distance units. A larger margin admits fewer candidates.

Algorithm 2: AUDITSTEP (w.p. p_a per query; background).

Input: graph G_t, corpus X; active level k_p, record depth K_{max}; uniform probability p_u; progress factor $\alpha > 1$; PQ margin $m_{pq} \ge 0$; audit settings from (5)

1 hold a consistent graph/layout state before drawing the target
2 $y \leftarrow$ w.p. p_u uniform from X, else a recent insert (uniform if that stratum is empty); retain stratum tag
3 $\mathcal{V} \leftarrow \textsc{BeamSearch}(G_t, x_y, L_{aud}, W)$
4 record one outcome: pass iff y examined or $\pi(y)$ fetched; release state
5 **if** *pass* **then return**
6 $U^* \leftarrow$ the few expanded nodes closest to x_y
7 **foreach** $u \in U^*$ **do**
8 **if** $\exists$ *reserve* v *of* u *with* $\text{dist}_{PQ}(v, x_y) \le \text{dist}(u, x_y)/\alpha - m_{pq}$ **then**
9 record attributable diagnostic
10 $\textsc{ProposeRepair}(u, v)$
11 **else**
12 record unattributable diagnostic; async: propose the best passing log candidate, if any; request pinning on repeated evidence
13 **end**
14 **end**

We calibrate the margin on held-out target–candidate pairs sampled by the audit procedure: measure $\max\{0, \text{dist}(v, x_y) - \text{dist}_{PQ}(v, x_y)\}$ and select a high percentile, reporting that percentile and margin. If the margin bounds underestimation for a tested pair, passing implies true α-progress; a sampled percentile does not establish this bound for every pair.

If a reserve passes, lines 9–10 record an *attributable* diagnostic and propose it for repair. Otherwise, line 12 records an *unattributable* diagnostic and searches the retained build log for the best progress-making candidate, if one exists. These per-node diagnostics are separate from the audit's single outcome. Logs remain after construction, compressed via Elias–Fano encoding [23], separately from the recorded build edges, as specified in Appendix I. Their storage and lookup I/O are additional maintenance costs (Section 11.6).

Proposals enter the node's bounded repair buffer $B_{miss}[u]$; uncached nodes use the bounded admission queue in Section 7. The learner applies the same gain threshold λ as for other candidates (Algorithm 4). Repeated audit evidence requests pinning; the request is honored only after a feasible installation under the rules in Appendix B.2. The governor can also restore build edges throughout the index, bypassing the gain gate. A higher level becomes active after these restorations finish, including at evicted nodes.

5.2 Changing protection

Algorithm 3 runs after each commit batch of B served queries (Algorithm 4), where the operator chooses B. Its inputs include the current protection level k_p, maximum level K_{max}, and fallback level k_p^s, called *last-safe*. The fallback starts at $k_p^{(0)}$ and records the level from which the most recent reduction was approved. A higher

requested level j is pending until restoration completes; k_{p} always denotes the active level. The counter *dwell* records served queries since the latest completed change or harm response.

The operator supplies the allowed miss rate ε_{r}, the additional margin ς, the redundancy and sole-witness tolerances δ_{red} and δ_{sw}, the repair-activity limit χ_{rep}, and the minimum wait T_{dwell} before a reduction. For build-edge length rank j, $\mathrm{red}[j]$ measures how often other retained edges provide comparable progress; $\mathrm{sw}[j]$ measures how often that edge alone provides observed progress. Appendix H defines their observations, windows, and the repair counter. The governor uses rank k_{p}, the boundary that a reduction would release, together with repair volume and the audit bounds below.

The governor computes LCB and UCB, the lower and upper confidence bounds on the uniform-target miss probability averaged over the states audited in the current window. Let m be the number of completed uniform-target audits and $\widehat{p}$ the fraction that missed. The operator chooses $\delta \in (0,1)$, the total probability budget for incorrect bounds across all checks and resets. Number audit windows by $w \geq 1$, counting each reset as a new window, and assign window w the budget $\delta_w = \delta/[w(w+1)]$. For $m \geq 1$, compute

$$a = \sqrt{\frac{\log(2m(m+1)/\delta_w)}{2m}},$$
$$\mathrm{LCB} = \max(0, \widehat{p} - a), \qquad \mathrm{UCB} = \min(1, \widehat{p} + a).$$

Here a is the uncertainty margin. A smaller δ widens the bounds and requires stronger evidence before lowering protection. Before any uniform audit completes, use $[\mathrm{LCB}, \mathrm{UCB}] = [0,1]$. Recent-insert audits do not enter these counts. In Appendix H, we prove validity under repeated checks and quantify the sample requirement; this conservative bound can delay reductions substantially.

Lines 1–2 of Algorithm 3 advance a pending promotion. Learning and layout commits pause while a bounded worker scans all records, restores required edges, and removes entries only outside the full recorded prefix $P_{K_{\max}}(u)$ and the pinned set. Queries continue. The worker activates the level only after all required writes are durable; Appendix B.2 gives the invariant and completion rule.

Lines 4–9 respond to harm. The requested level includes the current level, the fallback, and the build-edge ranks identified by diagnosis, so this branch cannot lower protection. If a fresh window still shows excessive misses at $K_{\max}$, the governor enters Freeze: learning stops, learner state is released, and queries continue on the retained index (Section 10). Lines 10–11 handle reductions after the wait and all three checks pass. A reduction releases a rank but removes no edges; replacement still requires the learner's gain threshold. Every completed level change starts a fresh audit window.

5.3 What the audit guarantee means

The governor may lower a positive protection level after the dwell interval T_{dwell} only if three checks pass: the audit bound satisfies $\mathrm{UCB} \leq \varepsilon_{\mathrm{r}} - \varsigma$; the boundary edges have sufficient observed redundancy, $\mathrm{red}[k_{\mathrm{p}}] \geq 1 - \delta_{\mathrm{red}}$; and their sole-witness rate is low, $\mathrm{sw}[k_{\mathrm{p}}] \leq \delta_{\mathrm{sw}}$. The margin ς, called *slack*, requires evidence below the allowed miss rate. These operator-set thresholds are defined in Section 5.2. Serving and learning continue during the wait.

Algorithm 3: Governor (every B queries, after Alg. 4).

Input: active level k_{p}, maximum $K_{\max}$, fallback $k_{\mathrm{p}}^{\mathrm{s}}$; pending promotion, if any; audit bounds $[\mathrm{LCB}, \mathrm{UCB}]$; $\mathrm{red}[k_{\mathrm{p}}]$, $\mathrm{sw}[k_{\mathrm{p}}]$, repair volume; ε_{r}, slack ς, tolerances $\delta_{\mathrm{red}}, \delta_{\mathrm{sw}}$, repair limit χ_{rep}, elapsed *dwell*, wait T_{dwell}

1 **if** *a promotion is pending* **then**
2 advance restoration scan; activate only on durable completion; **return**
3 **end**
4 **if** $\mathrm{LCB} > \varepsilon_{\mathrm{r}}$ *or repair volume* $> \chi_{\mathrm{rep}}$ *or an operator-supplied drift alarm* **then**
5 $j^{+} \leftarrow$ largest diagnosed reserve rank; if none, $\min(K_{\max}, k_{\mathrm{p}} + 1)$
6 $j \leftarrow \min\{K_{\max}, \max(k_{\mathrm{p}}, k_{\mathrm{p}}^{\mathrm{s}}, j^{+})\}$
7 **if** $j > k_{\mathrm{p}}$ **then** persist promotion request j; pause adaptive commits
8 **if** $k_{\mathrm{p}} = K_{\max}$ *and* $\mathrm{LCB} > \varepsilon_{\mathrm{r}}$ **then** enter Freeze; alert
9 reset statistics and *dwell*
10 **else if** *dwell* $\geq T_{\mathrm{dwell}}$, $k_{\mathrm{p}} > 0$, $\mathrm{UCB} \leq \varepsilon_{\mathrm{r}} - \varsigma$, $\mathrm{red}[k_{\mathrm{p}}] \geq 1 - \delta_{\mathrm{red}}$, *and* $\mathrm{sw}[k_{\mathrm{p}}] \leq \delta_{\mathrm{sw}}$ **then**
11 $k_{\mathrm{p}}^{\mathrm{s}} \leftarrow k_{\mathrm{p}}$; persist and activate $k_{\mathrm{p}} \leftarrow k_{\mathrm{p}} - 1$; reset statistics and *dwell*

The audit bound uses only completed uniform-target audits. Each draws a fresh target after the graph, layout, and search configuration are fixed for that audit, and evaluates a well-defined index state. Each audit contributes one outcome; per-node diagnostic records are not extra samples. The confidence construction in Section 5.2 accounts for repeated checks and resets within the total error budget δ [8].

Under these sampling conditions, with probability at least $1 - \delta$, every passing audit check certifies that the average true uniform-target miss probability over the audited states is at most $\varepsilon_{\mathrm{r}} - \varsigma$. Appendix H gives the full statement and proof, including how decreases in audit success probability since the earliest audit affect the bound for the current index.

6 LEARNING EDGE REPLACEMENTS

The learner uses query observations to identify useful edges, adjusts their scores for read cost, and commits a replacement only when its estimated gain justifies a write.

6.1 Estimating candidate utility

For each node u in the learner cache $\mathcal{H}$, the learner maintains a candidate pool $\mathcal{P}(u)$ of at most C identifiers, typically $2R$–$4R$, where R is the degree budget. The pool combines the current out-list with two buffers: $B_{nn}[u]$ holds candidates from query results and reverse-edge proposals, and $B_{\mathrm{miss}}[u]$ holds missed answers and audit repair candidates. When a query visits u, Algorithm 1 updates every candidate's utility estimate $\widehat{Q}(u,v)$ using $\mathbf{1}[v \in T_t]$: one if candidate v appears in the query's label set T_t, and zero otherwise. These labels come from the ordinary or probe search results. A candidate need not be a current neighbor of u; the update only tests its identifier for membership in T_t.

For a stationary workload, the learner uses a running mean with step $\beta_{u,v} = 1/(1 + n_{u,v})$, where $n_{u,v}$ counts observations already incorporated in that candidate's estimate. Each estimate has its own counter, initialized to zero on admission; samples start with subsequent queries. Cost estimates use their own availability-observation counters. Each new observation therefore has less influence as evidence accumulates. To follow a changing workload, the learner instead uses an exponentially weighted moving average (EWMA) with fixed rate β. After a further updates, an observation has weight $\beta(1-\beta)^a$, giving an effective history of roughly $1/\beta$ observations. Section 9 discusses how approximate labels affect the resulting ranking.

Adding (u, v) proposes u to the destination node's buffer $B_{nn}[v]$, using bounded admission when v is not cached (Section 7). The learner at v then evaluates the proposed reverse edge (v, u) using the same selection and commit rules as other candidates (Algorithm 4). For scoring, each query builds one hash set of its k labels, followed by $O(C)$ lookups per visited cached node. Membership tests require CPU work but no additional candidate-vector reads. With fixed pools and protection, retained learner state, exact labels, and ungated running-mean selection, A1–A3 imply that the preferred candidate sets eventually stabilize. We establish this result in Appendix B.

6.2 Accounting for I/O cost

A useful edge may lead to a block that is resident or has already been fetched. To estimate the extra read from node u, let $\mathcal{A}_{q,u}$ contain the blocks available when search is about to inspect u's neighbors, after fetching u's record. Define

$$c(u, v) = \mathbb{P}[\pi(v) \notin \mathcal{A}_{q,u} \mid u \in \mathcal{V}(q)]. \tag{6}$$

For cached candidates, search records this availability indicator at that moment; learning updates its mean $\widehat{c}(u, v)$ after the query. It does not use end-of-query availability, which would count a block already read by that query as free. The estimate starts at zero for a resident block and one otherwise. This source-specific proxy measures availability on observed visits; it does not predict how a new edge will change future routes.

The learner combines conditional utility $Q^\star(u, v)$ (Section 2.2) and read cost using a nonnegative weight θ:

$$Q_\theta(u, v) = Q^\star(u, v) - \theta\, c(u, v). \tag{7}$$

Its estimated score is $\widehat{Q}_\theta = \widehat{Q} - \theta\widehat{c}$. For candidates with similar utility, this score favors the one less likely to require a read. Higher utility can still justify a more costly edge.

The weight θ determines the preferred candidate set; the threshold λ determines whether the estimated improvement warrants a write (Section 6.3).

At each batch, the controller compares the measured mean number of serving reads $\widehat{I}$ with the operator's target $I^\star$ (Section 2.2) and updates θ:

$$\theta \;\leftarrow\; \left[\theta + \gamma\,(\widehat{I} - I^\star)\right]_+. \tag{8}$$

Here $\gamma > 0$ is the step size, and $[\,\cdot\,]_+$ clips negative values to zero. Above-target I/O raises the cost penalty; below-target I/O lowers it. The additive approximation $I(G) = \sum_u \mu(u) \sum_{v \in N_{\text{out}}(u)} c(u, v)$ weights each node's out-list cost by its visitation probability $\mu(u)$; $N_{\text{out}}(u)$ denotes that out-list. We write $\widehat{G}_\theta$ for a value-greedy fixed point under Q_θ: each node selects its highest-scoring feasible set under the traffic generated by that graph, as defined in Section 2.2. The following result separates selection under fixed reference measurements from control of measured serving I/O. In its controller statement, t indexes batch updates and $a > 0$ is a constant.

Theorem 6.1 (Cost-adjusted selection). *Fix the protected set, pools, per-node out-degrees, and reference measurements used to compare out-lists. Let $\bar{F}$ and $\bar{I}$ denote their additive utility and cost. Selecting the best feasible out-list at each node maximizes $\bar{F} - \theta\bar{I}$. For $\theta > 0$ the selection is Pareto-optimal for these two reference objectives; at $\theta = 0$ it maximizes $\bar{F}$. Its reference cost is non-increasing in θ. For the served-I/O controller, if $\gamma_t = a/t$, $a > 0$, and θ_t converges to a finite positive limit, then the time average of the measured batch I/O converges to $I^\star$.*

For edge selection, the theorem makes the trade-off precise. With the pools, protection, and reference measurements fixed, a positive θ selects out-lists for which reducing reference cost requires giving up utility, and increasing utility requires accepting more cost. This is Pareto optimality for the two reference objectives. Raising θ cannot increase the selected lists' reference cost. Section 6.3 determines whether to install those lists.

The controller uses this preference to respond to measured I/O. If the target is 20 block reads per query and a batch averages 24, it raises θ, making expensive candidates less attractive. When the measured average falls below the target, it lowers θ and gives utility more weight. With the stated a/t steps, if θ converges to a finite positive value, the average measured I/O across batches meets the target. Individual queries and batches may exceed it, and convergence of θ is a condition of the result.

The distinction between reference cost and measured I/O matters in practice. Rewiring can change search routes and block reuse, so a better reference score need not produce the same reduction in actual reads. The controller uses measurements from the adapted index to adjust subsequent choices. The full proof appears in Appendix C.

6.3 Deciding when to commit

The threshold λ specifies the minimum estimated gain required to replace learned edges. At each batch, Algorithm 4 selects the preferred set A for the $R_\ell(u)$ currently unprotected entries in node u's out-list. The quota is recomputed after protection changes, so a replacement preserves the degree. It compares the total cost-adjusted utility of A with that of the current learned set, excluding the protected edges $S(u)$. The difference is the estimated gain g. Shared entries cancel, so an unchanged selection has zero gain. If $g > \lambda$, the learner installs the protected set followed by A, denoted $S(u) \,\|\, A$, and queues the changed entries for writing.

Reverse-edge proposals undergo the same test at their destination. The write queue combines updates to the same block and processes them below read priority, subject to the configured limit on I/O operations per second (IOPS).

A small λ admits modest improvements and may cause frequent replacements as estimates change. A larger λ suppresses these writes but can retain a set with lower cost-adjusted utility. The threshold therefore determines how much estimated improvement the system requires before incurring a write.

Algorithm 4: CommitBatch (every B queries).

Input: hot cache $\mathcal{H}$; cost weight θ; step size γ; threshold λ; quantile q; I/O target $I^\star$

$\theta \leftarrow [\theta + \gamma(\widehat{I} - I^\star)]_+$ // dual ascent (8)
if *a promotion is pending* **then return**
foreach *unflagged* $u \in \mathcal{H}$ **do**
  $\mathcal{P}(u) \leftarrow N_{\text{out}}(u) \cup B_{nn}[u] \cup B_{\text{miss}}[u]$ (cap at C, retaining $N_{\text{out}}(u)$)
  $R_\ell(u) \leftarrow |N_{\text{out}}(u) \setminus S(u)|$
  $A \leftarrow \text{top}_{R_\ell(u)}\{\widehat{Q}[u][v] - \theta\widehat{c}(u,v) : v \in \mathcal{P}(u) \setminus S(u)\}$
  $g \leftarrow \sum_{v\in A} \widehat{Q}_\theta(u,v) - \sum_{v\in N_{\text{out}}(u)\setminus S(u)} \widehat{Q}_\theta(u,v)$
  if $g > \lambda$ **then**
    $N_{\text{out}}(u) \leftarrow S(u) \,\|\, A$; Enqueue(write of u's suffix)
    foreach *new* $v \in A$ **do** ProposeReverse(v,u) via bounded admission
  end
  honor feasible pin requests for installed edges, including unchanged lists
end
$\lambda \leftarrow \text{quant}_q\{|g(u)| : u \text{ with } A \neq \text{incumbent}\}$ // threshold for the next batch
publish queued changes in block order with bounded buffering, below read priority and against the IOPS ceiling

After each batch, we set λ for the next batch to the q-quantile of $|g(u)|$ over nodes whose proposed set differs from its pre-commit incumbent; the commit-disabled warm-up batch initializes this threshold. Thus q controls the gate using the previous batch's gain distribution. This adaptive operating rule is distinct from the fixed-λ setting of the write bound below.

For fixed λ, we bound the resulting maintenance work.

Theorem 6.2 (Write complexity; summary). *With fixed candidate pools and protection, a fixed threshold $\lambda > 0$, and the bounded-score conditions in Appendix D, the number of accepted replacements is bounded by the initial room for improvement plus accumulated changes in candidate scores, divided by λ. Under the running-mean and controller conditions in Appendix D, this count grows at most logarithmically with the number of queries.*

The bound counts accepted out-list replacements; physical writes also depend on queue coalescing and other maintenance work. Appendix D gives the full write bounds, including those for estimates that track workload changes, the threshold-selection conditions, and the proofs.

7 BOUNDING LEARNER MEMORY

Pagewire keeps learner statistics for at most H nodes in a least-frequently-used (LFU) cache with aging. Eviction removes statistics; the node retains its out-list and protection and continues to serve queries. Each cached node has at most C candidates and bounded proposal buffers. An operator-sized queue of J entries holds proposals for uncached nodes; it admits a node by evicting cache state, or drops the oldest unprocessed proposal when full. Flagged nodes are excluded. Thus learner and proposal memory is $O(HC + J)$, independent of corpus size; dropping a proposal can delay repair and carries no repair-completion guarantee.

Our analysis measures the candidate utility forgone relative to choosing the best feasible neighbors at each node, using the same search visits. It assumes independent reference queries with a fixed visitation and reward law, fixed candidate pools, replacement quotas, and cached set, exact labels, and running-mean learning without the write threshold. The following summarizes the memory bound.

Theorem 7.1 (Bounded-memory learning, summary). *Under this fixed reference model and a positive candidate-utility gap, learning at the H most frequently visited nodes incurs bounded expected cumulative candidate-utility loss at those nodes, independent of query count. Each expansion outside this set adds at most R_ℓ to the loss, where R_ℓ bounds the number of replaceable entries at any node.*

The result ties the memory budget to the search traffic it covers: learning at cached nodes has finite cumulative utility loss, while the remaining bound depends on how often search expands uncached nodes. This is a utility bound; it does not directly bound recall or read cost. The deployed aging cache can incur additional loss from admission errors and relearning after eviction. Additional memory cannot resolve candidate rankings at nodes that receive too few observations. Appendix E gives the full bounds, memory-sizing analysis, sample requirements for rarely visited nodes, and proofs.

8 REORGANIZING THE BLOCK LAYOUT

Edge selection determines which nodes search visits. Layout determines how many block reads those visits require. If a query visits several nodes in the same block, one read can serve all of them. The layout problem is to place nodes in blocks of capacity κ so that queries touch fewer blocks, without changing the graph edges. To isolate the effect of placement, we first consider a fixed distribution of expanded-node traces $\mathcal{V}(q)$, with no blocks resident before a query and each fetched block available for the rest of that query. The objective is to minimize the expected number of distinct blocks read, or equivalently to maximize reads saved relative to reading each visited node separately.

Choosing which nodes to place together. Pagewire uses pairwise co-visitation to guide placement. Let $\text{co}(u,v) = \mathbb{P}_q[u \in \mathcal{V}(q) \wedge v \in \mathcal{V}(q)]$ be the probability that a query expands both u and v. For a layout π, the pairwise objective is

$$W(\pi) = \sum_{\{u,v\}:\pi(u)=\pi(v)} \text{co}(u,v).$$

We maximize this weight under the block-capacity constraint, counting each unordered pair once. Each node in the learner cache keeps a bounded sketch of frequently co-visited partners, updated from served queries.

Improving the layout incrementally. When no promotion is pending, every B queries, LayoutStep (Algorithm 1) considers moving a partner into a node's block and displacing an occupant with low co-visitation with its blockmates. It evaluates the change in total pairwise weight across the affected blocks and accepts the swap only if the estimated gain exceeds λ_π, the minimum gain required for a layout change. The two sector writes enter the maintenance queue; the relocation map publishes the swap through the journaled protocol in Appendix I. Moving records leaves the graph edges, including the protected set, intact.

Relating the objective to saved reads. Larger groups can make pairwise weight overstate the savings. Suppose a trace visits three nodes. Keeping them in separate uncached blocks takes three reads; placing them in one block takes one and saves two. The pairwise score counts three pairs. Thus the score can overcount the reads saved when more than two visited nodes share a block.

Block capacity bounds this overcount. If a trace visits z nodes in one block, it saves $(z-1)_+$ reads and contributes $\binom{z}{2}$ visited pairs, where $(z-1)_+ = \max\{z-1, 0\}$. Since $z \le \kappa$,

$$\frac{2}{\kappa}\binom{z}{2} \le (z-1)_+ \le \binom{z}{2}.$$

Summing over blocks and averaging over queries shows that expected savings lie between $2/\kappa$ times $W(\pi)$ and $W(\pi)$. Consequently, a layout attaining a fraction ρ, with $0 < \rho \le 1$, of the largest possible pairwise weight saves at least $2\rho/\kappa$ of the reads saved by the best layout. This compares savings under the same trace distribution. For example, at capacity four, an exact maximizer of pairwise weight achieves at least half of the best possible savings.

With two nodes per block, pairwise weight and savings coincide. Finding the best layout then amounts to choosing disjoint pairs with the largest total co-visitation weight, allowing unpaired nodes to remain alone. This is maximum-weight matching and can be solved exactly. For larger blocks, a higher pairwise score need not mean fewer reads. The savings bound applies once the fraction ρ of optimal pairwise weight is established; accepting improving swaps alone does not establish that fraction.

Layout during graph adaptation. With fixed co-visitation weights, strictly improving swaps eventually stop because no layout can be revisited. If every positive-gain swap is eligible and the procedure stops only when none remains, the final layout is a local optimum. Pagewire examines recorded partners and requires a gain above λ_π; exhausting those swaps rules out only gains above that threshold among eligible partners at the final layout. During adaptation, rewiring changes co-visitation and relocation changes read costs, so this fixed-measurement result does not establish that the combined process eventually stops. Appendix F gives the full statements and proofs for the savings bound and the termination of improving swaps.

9 LEARNING FROM APPROXIMATE RESULTS

Pagewire learns from approximate answers, so systematic label errors can distort candidate rankings. Algorithm 1 independently selects each query for a higher-effort probe with probability p. The probe result replaces the ordinary learning label; answers found only by the probe enter the repair-candidate buffer B_{miss}.

For a fixed reference graph, candidate pools, and costs, we show that running-mean estimates eventually identify the same preferred candidate set as exact labels if the systematic error in each candidate's utility is less than half the cost-adjusted score gap between selected and excluded candidates. This condition accounts for both label accuracy and the probe probability p; occasional accurate probes alone do not ensure the correct ranking. We also establish when the gain from replacing an incorrect current selection exceeds the commit threshold despite label bias and sampling error.

In Appendix G, we relate probe accuracy to candidate-utility bias and give the full conditions and proofs.

Table 2: Parameters, initial settings, and corresponding results.

Resource	Control	Default	Analysis
Degree	k_{p} (governed)	$k_{\text{p}}^{(0)}$ (§4)	Thm. 4.2
Record	$K_{\max}$	calibrated, capped at R	App. B.2
I/O	$I^\star$ (drives θ)	SLO value	Thm. 6.1
Writes	λ	calibrated in §6.3	Thm. 6.2
Writes	q (drives λ)	tested in §11.7	—
Memory	H	memory budget	Thm. 7.1
Layout	λ_π	2λ	App. F
Labels	p, L_{pr}	10^{-3}–10^{-2}, $10L$	App. G
Audits	p_{a}, ε_{r}	10^{-3}–10^{-2}; $2\widehat{b}_0{+}10^{-3}$	App. H
Audit mix	p_{u}	0.9	§5.1
Dwell	T_{dwell}	operator-set wait	App. H
Batch, Pool	B; C	100; $2R$–$4R$	—

Table 3: Vector-search datasets. ESCI uses real product-search queries [19].

Dataset	Description	Dim.	Data type	Objective
SIFT-10M	Local-image descriptors	128	uint8	ℓ_2
DEEP-10M	Deep image embeddings	96	float	ℓ_2
T2I-100M	Text-to-image embeddings	200	float	MIPS
SpaceV1B	Web-text embeddings	100	int8	ℓ_2
ESCI-1.44M	Product embeddings	384	float	ℓ_2

10 DEPLOYMENT

Table 2 summarizes the parameters and initial settings. To initialize Pagewire, estimate the baseline miss rate $\widehat{b}_0$ through offline uniform-target audits of G_0 with the deployment audit configuration. Choose $I^\star$ and the estimator regime, initialize costs from residency, and run one or two batches with commits disabled and $\theta = 0$ to warm estimates and measure baseline I/O and mean expansions per query $\bar{v}_0$. Calibrate λ as in Section 6.3, then start the controller at $\theta = 0$ with $\gamma_b = 0.1/(\bar{v}_0 b)$, where $b \ge 1$ counts controller updates and $\bar{v}_0$ remains fixed after warm-up. Appendix I provides further parameter tuning, background-I/O accounting, concurrency and recovery protocols, operating modes, and compatibility details.

11 EXPERIMENTAL EVALUATION

11.1 Experimental Setup

We evaluate five vector-search datasets (Table 3). For the synthetic workloads, we add Gaussian noise with standard deviation σ to related base vectors; smaller σ concentrates queries. Unless stated otherwise, the block cache $\mathcal{C}_{\text{blk}}$ has a byte budget equal to 2.5% of the on-disk index size: on SIFT-10M, 62,500 sectors with four nodes per sector. The learner-state cache $\mathcal{H}$ holds at most $H = 16{,}384$ nodes.

Experiments use an AWS instance with a 3.5 GHz Intel Xeon Ice Lake processor, 247 GB memory, and local NVMe storage. SpaceV1B runs on a shared node with 2× AMD EPYC 9654 processors, an allocation of 64 cores and 1.2 TB DRAM, and a local NVMe SSD. All serving runs use a single thread. We use 32-byte PQ codes, $R = 64$, and a separate mutable index copy for each run. Direct I/O (`O_DIRECT` through `libaio`, with `io_uring` disabled) bypasses the OS page cache. Paired comparisons share the initial index, query order, and serving parameters ($L = 100$, $W = 8$). Unless stated

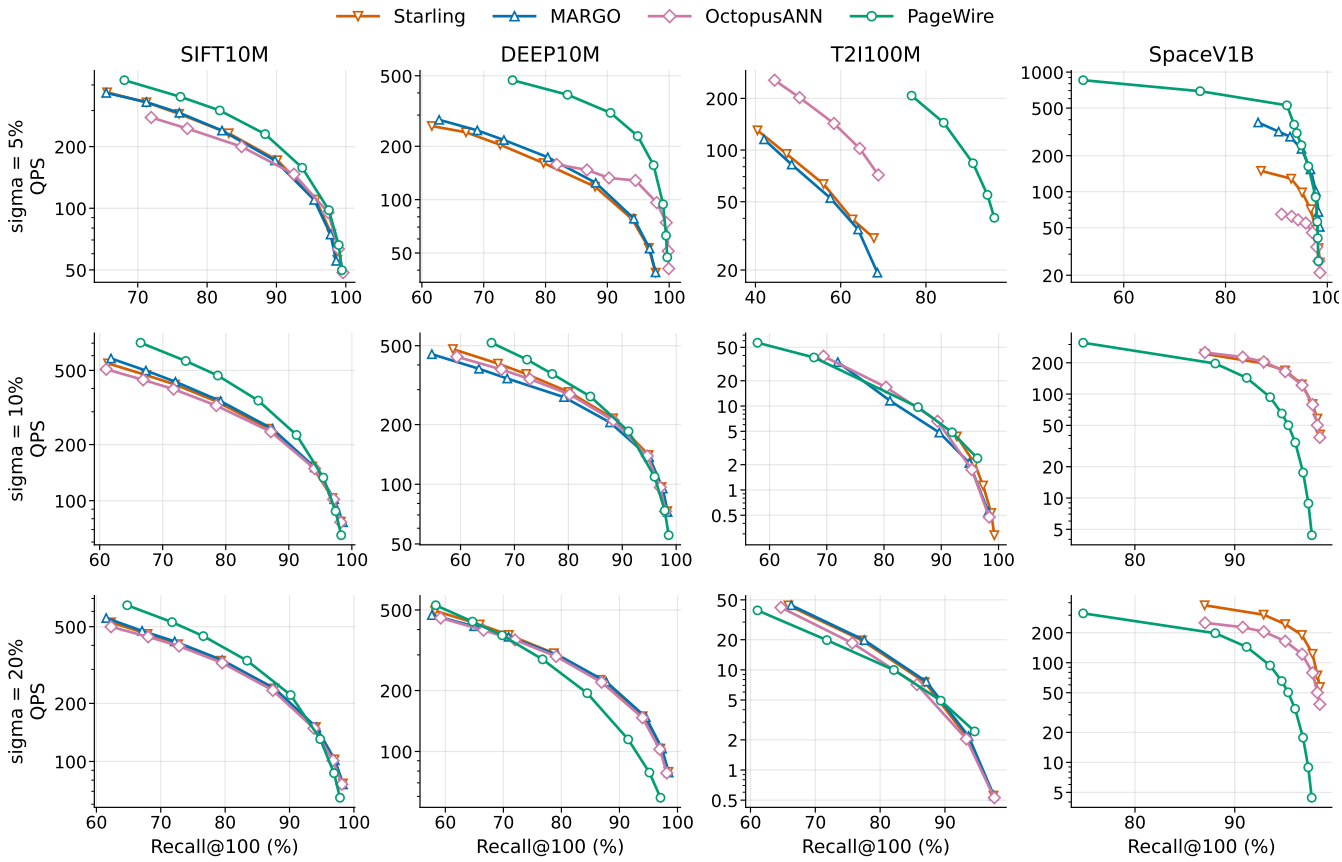


Figure 1: Recall@100 versus throughput across four datasets and three workload-noise levels ($\sigma = 5\%, 10\%, 20\%$, top to bottom). Each curve sweeps search breadth after adaptation or baseline preprocessing.

otherwise, adaptation uses 5,000 randomly permuted queries with period $B = 100$; we report Recall@100 and queries per second (QPS). By default, T_t contains approximate top-100 results, replaced by higher-effort probe results when selected (Section 9). Rewards are binary, $\mathbf{1}[v \in T_t]$, with full-information updates over each visited cached node's candidate pool, including candidates outside the out-list (Section 6.1).

We first compare serving performance (Section 11.2), isolate graph and layout gains (Section 11.3), and vary workload dispersion (Section 11.4). We then evaluate label-rate sensitivity, route protection, I/O control, learner-cache capacity, and workload drift (Sections 11.5–11.9).

11.2 Baseline Comparison

We compare PageWire with Starling [24], which improves block locality; MARGO [25], which uses monotonic-path edges; and OctopusANN [17], which combines memory-resident navigation with page-level and dynamic-width search. All systems start from the same initial graph. We evaluate PageWire after adaptation and freezing, and each baseline after its prescribed preprocessing. We sweep search breadth under the same hardware, concurrency, and direct-I/O settings. Within each dataset and noise level, systems are evaluated on the same held-out queries at $\sigma \in \{5\%, 10\%, 20\%\}$; Section 11.4 examines adaptation recall over a broader dispersion range.

Figure 1 shows the clearest recall–throughput advantage for PageWire at $\sigma = 5\%$, particularly on DEEP-10M and T2I-100M. As noise rises to 10% and 20%, the advantage narrows and sometimes reverses; SIFT-10M remains comparatively strong, whereas DEEP-10M, T2I-100M, and SpaceV1B lose ground at higher noise. This trend is expected: more dispersed queries revisit fewer of the paths whose edges and blocks PageWire specializes. The next experiment separates the contributions of graph and layout adaptation.

11.3 System Ablation

Our 2×2 ablation compares **Static** (neither mechanism), **Layout** (layout only), **Rewire** (rewiring with protection and audits), and **Full** (both). Variants share the initial DiskANN index, adaptation queries, seeds, block-cache capacity, and learner cap H. We freeze each adapted index and evaluate recall and QPS on queries disjoint from adaptation.

Figure 2 shows complementary gains. On SIFT-10M, Layout increases throughput over Static, and Full improves further. On DEEP-10M, rewiring drives the largest shift toward higher recall;

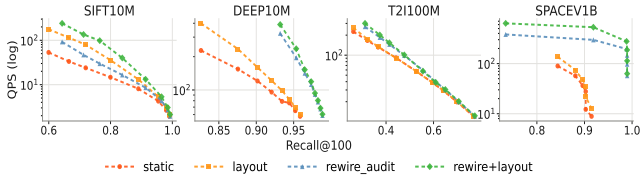


Figure 2: Graph and layout ablation on four datasets.

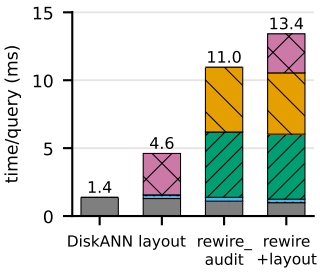


Figure 3: SIFT-10M adaptation time per query.

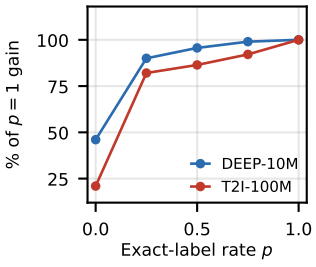


Figure 4: Recall gain relative to full exact labeling ($p = 1$).

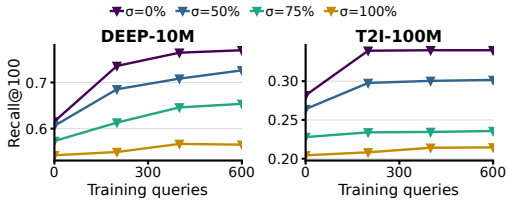


Figure 5: Held-out Recall@100 under workload dispersion.

layout adds a throughput benefit. On SpaceV1B, Full combines Rewire's recall advantage with higher throughput. On T2I-100M, Layout nearly overlaps Static and Full closely tracks Rewire, so graph adaptation accounts for most of the improvement. Its three nodes per sector limit the available co-location opportunities, although capacity alone does not determine savings (Section 8).

Figure 3 breaks adaptation time into beam search (gray), Q updates (blue), rewiring computation (green), writes (orange), and layout (purple). DiskANN takes 1.4 ms/query; Layout takes 4.6, Rewire 11.0, and Full 13.4. Rewiring computation and writes dominate the overhead. Freezing stops maintenance while retaining the adapted index; Section 11.9 tests this online.

11.4 Sensitivity to Workload Dispersion

To test sensitivity to specialization, we vary Gaussian-noise scale relative to the diameter of the provided query set, keeping the selected clusters and adaptation configuration fixed within each dataset. Figure 5 shows positive held-out recall gains at every plotted noise level. On DEEP-10M, zero-noise recall rises from approximately 0.61 to 0.77, and the 75% noise workload rises from 0.57 to 0.65. On T2I-100M, zero-noise recall rises from approximately 0.28 to 0.34, with smaller gains at higher dispersion. Adaptation thus benefits dispersed workloads as well, but concentration determines much of the gain.

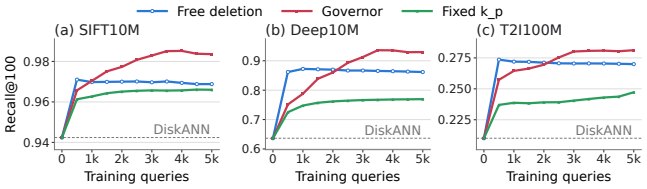


Figure 6: Training recall under three edge-protection policies.

11.5 Oracle Label-Rate Sensitivity

Approximate labels can misrank edges (Section 9). We self-label every query and additionally union exact top-100 answers into T_t with probability $p \in \{0, 0.25, 0.5, 0.75, 1\}$; $p = 1$ is an offline oracle. On $\sigma = 0\%$ DEEP-10M and T2I-100M, we use $k = L = 100$, $\lambda = \theta = 0$, and the governor protection settings of Section 11.6. Recall gain subtracts a separate untrained DiskANN control for each workload. Testing uses 2,000 held-out queries.

At $p = 0.25$, Figure 4 reaches 90% of the oracle gain on DEEP-10M and 82% on the plotted T2I-100M subset; at $p = 0.5$, the fractions are 96% and 86%. Thus sparse exact labels capture much of the gain in these samples, indicating efficiency of self-feedback.

11.6 Preserving Routes During Specialization

We next evaluate the mechanisms that constrain specialization: governed protection and repair on held-out out-of-distribution (OOD) queries. These recall measurements complement the theory in Sections 4 and 5.3; they do not certify global navigability.

11.6.1 Governed Edge Protection. The three policies share the graph, query stream, learner, degree budget, and rewire schedule. Free deletion may replace any outgoing edge; fixed protection holds $k_p = k_p^{(0)}$ at the build-time calibrated initial level (Section 4); governed protection adjusts k_p using the governor in Section 5.2 with the experimental settings below. This ablation measures the recall benefit of adaptive protection. We use $p_a = 1$, $p_u = 0.9$, $L_{aud} = 1$, $T_{dwell} = 200$ queries, $\varsigma = 0.005$, and $\varepsilon_r = 0.11$. As in Section 5.1, the empty recent-insert stratum makes all audit targets uniform samples from the full corpus.

Figure 6 shows the governor outperforming both alternatives in final recall on all three datasets. Free deletion improves quickly but plateaus below the governor; fixed protection restricts specialization. On DEEP-10M, final recall is approximately 0.93 with the governor, 0.86 with free deletion, and 0.77 with fixed protection—advantages of about 7 and 16 percentage points. Across these workloads, adaptive protection achieves higher final recall than either fixed protection or unrestricted deletion.

11.6.2 Audit Repair and Overhead. We isolate repair using 2,000 uniformly sampled held-out OOD queries ($k = 100$, $L = 100$). Without injected damage, recall does not degrade beyond run-to-run noise. We therefore remove half the outgoing edges of each node

Table 4: Audit-repair isolation: held-out OOD Recall@100 after injected edge damage. Repair-off recall equals the post-damage starting value.

Dataset	Undamaged	Repair off	Repair on
DEEP-10M	0.7104	0.4700	**0.5675**
SIFT-10M	0.7221	0.3700	**0.4788**
T2I-100M	0.5613	0.3725	**0.4292**

Table 5: Candidate-log storage. Parentheses give the fraction of index size.

	SIFT-10M	DEEP-10M	T2I-100M
Index (GB)	9.1	6.83	136.5
Raw log (GB)	6.8 (74.7%)	6.82 (99.9%)	68.690 (50.3%)
Encoded (GB)	2.8 (30.8%)	2.7 (39.5%)	28.231 (20.7%)

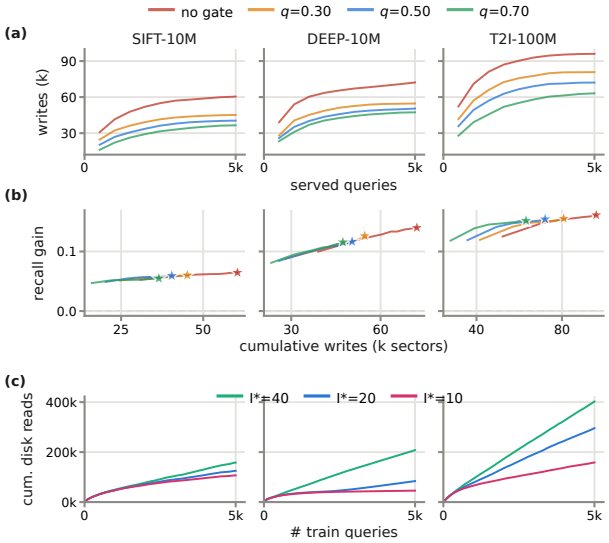


Figure 7: I/O controls: (a) rewire writes; (b) recall gain versus writes; (c) traversal reads versus $I^\star$.

entering the learner cache, including protected edges, disable ordinary proposals from B_{nn} and B_{miss}, and retain only audit-repair proposals in B_{miss}.

Table 4 shows gains of 9.75, 10.88, and 5.67 percentage points on DEEP-10M, SIFT-10M, and T2I-100M, respectively. Repair recovers 40.6%, 30.9%, and 30.0% of the induced loss. This demonstrates recovery under deliberate damage, not that ordinary adaptation causes comparable losses.

Across DEEP-10M, SIFT-10M, and T2I-100M, auditing accounts for less than 2% of adaptation time in each reported 5,000-query run, and fewer than 2% of audits consult the retained construction-time candidate log. This fallback for *unattributable* misses lies outside the common serving path. Elias–Fano encoding [23] of sorted candidate identifiers, with an offset directory for random access, reduces log storage by approximately 60% (Table 5). The encoded log occupies 20.7–39.5% of index size.

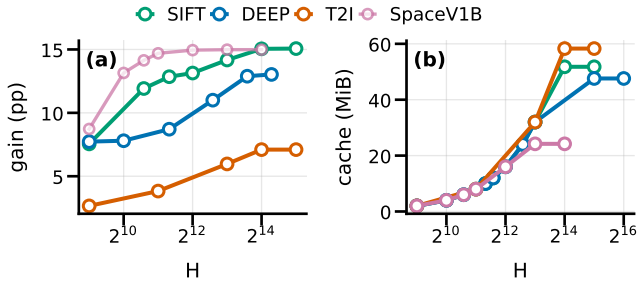


Figure 8: Effect of learner-cache capacity H: (a) Recall@100 gain over DiskANN; (b) realized cache size.

11.7 Cost-Aware Rewiring

We now examine maintenance writes and traversal reads. The gain gate λ filters commits; the target $I^\star$ drives the read-cost weight θ through Equation (8). Runs use 5,000 IID training queries and unidirectional rewiring with $B = 100$. Held-out recall uses 500 disjoint queries.

For write control, we vary $q \in \{0.30, 0.50, 0.70\}$, the previous-batch gain quantile defining λ (Section 6.3), and hold the I/O target fixed, with $\lambda_\pi = 2\lambda$. Two warm-up batches disable commits and use $\theta = 0$; subsequent controller updates use $\gamma_b = 0.1/(\bar{v}_0 b)$ as in Section 10. The block cache holds 32,768 sectors with LRU replacement and write-through updates. Physical rewire-write counts exclude forced restorations; they differ from the accepted replacements bounded in Theorem 6.2.

On SIFT-10M, the plotted $q = 0.70$ calibration reduces writes by over 40% while retaining most of the recall gain (Figure 7(a–b)). DEEP-10M and T2I-100M show the same trade-off. Gated curves flatten toward the end and remain below the ungated curves, demonstrating sustained savings over the measured horizon, rather than establishing asymptotic write elimination.

For read control, we set $\lambda = 0$, initialize $\theta = 0$, and sweep $I^\star \in \{10, 20, 40\}$ sectors/query. Figure 7(c) shows that smaller targets reduce cumulative traversal reads on all three datasets and hence average reads over the equal-length runs.

11.8 Memory Ablation

We vary the learner-state cache capacity H (Section 11); $\mathcal{H}$ holds the per-node Q-score state updated by the learner. A larger $\mathcal{H}$ lets more nodes retain update history between visits. We measure how this changes recall, memory use, and adaptation cost.

All datasets use 2,000 training queries, 1,000 held-out post-test queries, with otherwise the configuration of Section 11.3, so H is the only swept quantity within each workload. Panel (b) shows that the required learner-state memory does not increase with corpus size: the realized number of distinct nodes resident in $\mathcal{H}$, converted to MiB at 4,096 bytes per node. At saturation, the footprints are 51.8 MiB (SIFT-10M), 47.6 MiB (DEEP-10M), 58.3 MiB (T2I-100M), and 24.2 MiB (SpaceV1B). They reflect how many nodes the fixed 2,000-query stream touches on each graph. Once H exceeds that footprint, there is nothing further to admit; for example, SIFT-10M's $H = 16{,}384$ and $H = 32{,}768$ runs coincide in realized size and recall. Device reads count misses in C_{blk}; evicting Q-state from $\mathcal{H}$ merely

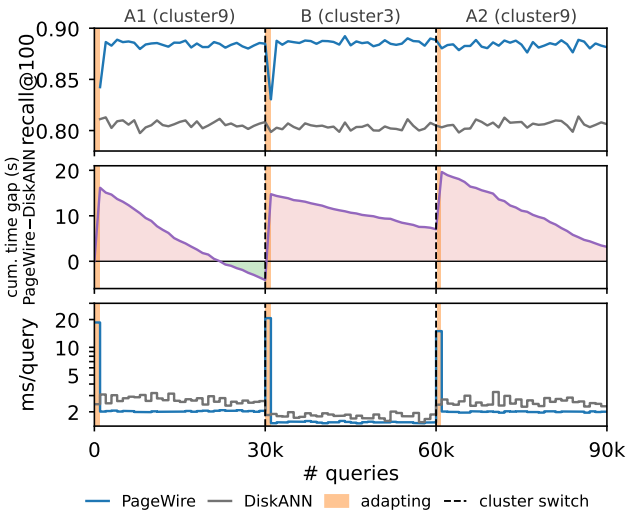


Figure 9: ESCI *A–B–A* drift (30,000 queries per phase): Recall@100, cumulative runtime gap versus DiskANN, and query latency (top to bottom). Shading marks adaptation; dashed lines mark switches.

reinitializes that state in RAM. Device writes count committed sector updates, not evictions from $\mathcal{H}$.

Figure 8(a) shows recall gain increasing with H on every dataset, rising rapidly at small capacities and approaching a plateau near or by $H = 2^{14}$. Choosing H between 2^{14} and 2^{15} captures nearly all the measured recall gain. Panel (b) shows that the required learner-state memory does not increase with corpus size: SpaceV1B's much larger collection reaches a roughly 24 MiB footprint, within the same modest range as the 48–58 MiB footprints of the smaller datasets. The local workload, rather than the number of indexed vectors, determines how many nodes need retained state.

Adaptation latency varies within measurement noise as H increases. Because $\mathcal{H}$ holds only in-memory Q-state, measured device reads per query remain nearly flat. Additional rewire commits contribute little time beside these search costs, so H acts primarily on recall and rewire-write work rather than latency.

11.9 Adaptation Under Workload Drift

We evaluate automatic freezing during stable traffic and renewed adaptation after workload shifts on ESCI (Table 3). We assign queries to 14 categories adapted from Amazon's browse-node taxonomy using keyword/title matches and nearest-centroid fallback in ESCI-fine-tuned MiniLM embeddings[1]. Cluster 9 (lawn-mower/tire) forms A, and cluster 3 (tools/hardware) forms B. The stream follows an A–B–A sequence with 30,000 queries per phase. The experiment uses $R = 32$, $k = L = 100$, $B = 500$, self-labels, and a 24,576-sector (96 MiB) block cache for both systems.

Utility estimates use an EWMA with fixed $\beta = 10^{-2}$ (Section 6.1). Protection and audits use Section 11.6's settings. The controller evaluates 100-query windows. It freezes when Δ_Q, the mean absolute change in $\widehat{Q}(u, v)$ over the window, falls below the 20th percentile of Δ_Q values from the latest B-query batch ($B = 500$). This uses utility changes already tracked by the learner to identify stable traffic. While frozen, it samples one query in 50 and restarts adaptation when over 70% of sampled candidate edges are absent from the edge signature saved at the last freeze. Together, utility changes and sparse edge-signature probes provide freeze and restart signals without a separately trained drift detector. Utility estimates are reset to the initial prior k/n on restart.

Figure 9 tracks Recall@100, cumulative runtime relative to DiskANN without rewiring, and per-query latency. We also evaluate 1,000 queries sampled uniformly at random from the dataset's query set to assess whether adaptation degrades out-of-distribution (OOD) recall.

Results show that PageWire adapts within 800 queries in each phase, reaching recall around 0.88 versus DiskANN's 0.80–0.81 (top panel). Adaptation causes brief runtime spikes, but the cumulative runtime overhead steadily declines during subsequent serving (middle panel): outside adaptation, PageWire's query latency is approximately 20–25% lower on average (bottom panel). Recall changes on the random-query evaluation remain within the noise level, indicating no measurable OOD degradation in this workload.

12 RELATED WORK

Graph construction. DiskANN [22] combines a disk-resident Vamana graph with in-memory compressed vectors. NSG [6] balances connectivity, out-degree, and path length; HNSW [18] uses hierarchical search. Pagewire adapts the out-lists of a deployed Vamana graph while retaining selected build edges.

Disk layout. Starling [24] improves useful work per I/O through layout and search techniques. MARGO [25] uses monotonic-path edge importance to organize records. Pagewire instead observes served-query co-visitation to propose incremental block swaps, sharing maintenance resources with rewiring.

Query-aware and evolving indexes. OOD-DiskANN [12] and RoarGraph [5] use queries to guide construction. FreshDiskANN maintains a graph under vector insertions and deletions [21]. Pagewire studies changing query traffic over a fixed collection, adapting an existing index during serving.

Adaptive physical design. Cost-driven index selection [4], online physical-design tuning [3], and database cracking [7, 10] reorganize data for observed workloads, accounting for adaptation cost or query order. Pagewire makes these decisions for individual graph edges and disk blocks while preserving routes needed by less frequent queries. Cost-aware object retrieval [1] also balances usefulness against access cost.

Analysis tools. Graph-ANN and proximity-search analyses [11, 15] relate geometric progress to search. Our learning analysis uses stochastic approximation [20] and gap-dependent selection [2, 16]; audits use confidence sequences [8], and layout analysis uses a union-probability bound [9].

[1] https://huggingface.co/metarank/esci-MiniLM-L6-v2

13 CONCLUSION

Pagewire adapts disk-resident graph edges and record placement using served queries, with protected build edges, audit repair, bounded learner state, and cost-aware commits. The evaluation shows complementary graph/layout gains, higher final recall with governed protection, and lower write and read costs under the respective controls. Under workload drift, Pagewire resumes adaptation and retains recall gains after freezing.

A ASSUMPTIONS AND ANALYTICAL SCOPE

For fixed pools and protection, write $\mathcal{E}(u) = \mathcal{P}(u) \setminus S(u)$ for the eligible candidates and $R_\ell(u)$ for the fixed replacement quota. Nodes with zero quota, or with no excluded eligible candidate, require no ranking decision. All gaps below concern the other analyzed nodes. There are no external graph, layout, pool, or protection changes during these learning intervals.

Assumption A.1 (A1: stationary regular workload). *Each query is drawn independently of the past from a fixed $\mathcal{D}$. The graph used for that query is determined by the past. For every $u \in V_+$ and every reachable G, $\mu_G(u) \geq \mu_{\min} > 0$. At successive visits to u, exact-label observations have conditional means $Q^\star_G(u, v)$ given the past and the visit. Their centered values are bounded martingale differences; independence after conditioning on the entire adaptive graph history is not assumed.*

Assumption A.2 (A2: eligible-candidate gap). *Let $\Delta^\flat$ be the infimum, over analyzed nodes and reachable graphs, of the true-utility difference between the last candidate selected into the top-$R_\ell(u)$ set in $\mathcal{E}(u)$ and the first excluded candidate. Assume $\Delta^\flat \geq \Delta > 0$. Ties away from this boundary use identifiers.*

Assumption A.3 (A3: bounded visitation drift). *There is $\eta < \Delta^\flat/2$ such that $|Q^\star_G(u, v) - Q^\star_{G'}(u, v)| \leq \eta$ for all reachable G, G', analyzed nodes u, and eligible candidates $v \in \mathcal{E}(u)$.*

A1 ensures observations; A2 separates candidates; A3 limits ordering changes. A3 is sufficient, not necessary, for the fixed-point argument. Exact-label results use an ideal observation model; Appendix G treats the actual mixture.

The geometric results assume $\alpha > 1$, metric distances, and at least two distinct vectors, with $d_{\min} > 0$, diameter D_X, and $\Delta_X = D_X/d_{\min} \geq 1$. Identical-vector ties require a corresponding equivalence-class convention. None of these assumptions converts quantized beam search into exact greedy search.

Remark A.4 (Coverage and multiplicity). For fixed traces, counting whether an answer occurs in any visited adjacency list is a coverage objective, while F counts occurrences with multiplicity. These are different from actual returned recall: an encountered identifier need not be expanded or returned, and a different graph can change the trace. The separable selection argument therefore compares out-lists under fixed reference measurements.

B PROPERTIES OF THE PROTECTED PREFIX

B.1 Candidate logs and certification

The log C_u is collected by Pagewire during Vamana construction [22, Section 2.3, Algorithm 3]. It contains pruning candidates from insertion searches and reverse-edge processing. Logging and certification are Pagewire additions. Certification tests whether the final prefix $P_j(u)$ covers logged candidates beyond $r_u(j)$. The full-list check also records build holes; intermediate pruning outputs do not certify the final retained list.

Log faithfulness means that, for every u and radius r, any edge set covering $\{z \in C_u : \mathrm{dist}(u, z) > r\}$ also covers $\{y \in X \setminus \{u\} : \mathrm{dist}(u, y) > r\}$. It is an additional hypothesis, not a consequence of a standard degree-capped build. Uniform corpus audits can expose search failures outside the logged candidates, but do not prove log faithfulness.

We also need to distinguish preserved distant progress from coverage at shorter distances. Certification does not require the protected prefix to cover targets at or within its landing radius; these targets may rely on edges the learner can replace. For the analysis, we call a certified node *violating* if its current full out-list leaves at least one such logged target without a progress witness. At a node whose active prefix is uncertified, including a flagged node, we call the node violating if its current full out-list leaves any logged target without a witness. Thus every node is classified at its active level; an unflagged node need not be certified at every possible level.

Thus, "violating" identifies a gap in progress coverage, rather than a failure to retain protected edges. A node can retain every protected edge and still be violating. Such gaps may exist in the original build or arise during adaptation. The label *violating* is used in the analysis. This distinction also matters when comparing with Vamana. Its ideal all-target guarantee uses full-corpus candidates without a restrictive degree cap [22, Section 2.2]; the practical construction does not establish unconditional global α-navigability. Pagewire's checks identify the progress that its protected prefixes provide on logged candidates, and Theorem B.5 states when that preservation extends to the corpus.

Proof of Theorem 4.2. Fix a certified prefix $P_j(u)$ and a logged target y with $\mathrm{dist}(u, y) > r_u(j)$. Certification supplies an edge (u, v) in that prefix with $\mathrm{dist}(v, y) \leq \mathrm{dist}(u, y)/\alpha$. Retaining the prefix retains this witness, regardless of replacements among the other edges. This proves the logged-target statement without requiring faithfulness. Under faithfulness, applying the condition to $P_j(u)$ and radius $r_u(j)$ extends its coverage to all corpus targets beyond that radius. □

B.2 Protection levels and initialization

Choosing an initial level. For integers $1 \leq j \leq R$, let $\mathrm{Cov}(j)$ be the fraction of nodes whose prefix $P_j(u)$ covers logged targets beyond $r_u(j)$. For an operator tolerance $\varepsilon_{\mathrm{cov}} \in [0, 1)$, let $j_0 = \min\{j \in \{1, \ldots, R\} : \mathrm{Cov}(j) \geq 1 - \varepsilon_{\mathrm{cov}}\}$. If this set is nonempty, choose

$$k_{\mathrm{p}}^{(0)} = \min(R, j_0 + 2), \qquad K_{\max} = \min\{R, \max(2k_{\mathrm{p}}^{(0)}, \lceil R/4 \rceil)\}.$$

If it is empty, set $k_{\mathrm{p}}^{(0)} = K_{\max} = R$ and report the failed calibration. The two-rank margin is an operating heuristic, not a coverage theorem. Check each actual level $0 \leq j \leq K_{\max}$ and store its certification bit; level zero is vacuous and is excluded from calibration. Flag nodes that fail at $K_{\max}$ and retain their full build lists. An unflagged node can have an uncertified prefix at another level; the preservation theorem applies only to its certified levels. Offline audits measure $\widehat{b}_0$.

Changing the level. The active level k_{p} and a pending requested level $j > k_{\mathrm{p}}$ are distinct durable fields. A request first pauses learned replacements, pin changes, layout moves, and corpus updates, and drains earlier accepted writes. Queries and audits continue. A worker scans the build records, including nodes evicted from the learner cache, using bounded memory. At each node it restores missing edges in $P_j(u)$, longest first, and removes entries

only outside $P_{K_{\max}}(u) \cup \mathrm{pin}(u)$ to keep the same out-degree. Thus restoration never removes another recorded build edge or a pin. The scan may instead use a complete disk-resident index of outstanding claims. Flagged lists remain unchanged. This scan and its writes are maintenance costs, not operations limited to the hot cache.

Until the scan finishes, the old active level remains in force. Only after all required writes are durable does the worker atomically publish j as active, clear the pending request, reset statistics and dwell, and resume adaptive commits. On restart, a pending scan is resumed or repeated before adaptation resumes; its updates are idempotent. Demotion atomically lowers the active level and resets the window without removing edges. Certification status is always read for the active level.

Pin installation. Repeated evidence means at least two distinct completed audits proposing the same (u, v) while its bounded proposal entry is retained; eviction resets this count. A request at an unflagged node is eligible only if

$$|P_{K_{\max}}(u) \cup \mathrm{pin}(u) \cup \{v\}| \le |N^{(0)}_{\mathrm{out}}(u)|.$$

This reserves enough degree for every future recorded level. If v is absent, it must first pass the ordinary replacement gate. An installed eligible edge can be pinned at the next batch even if no further replacement is needed. Persist the adjacency and pin metadata together before publishing the pin; defer requests during promotion. Ineligible requests remain diagnostic evidence but do not change the protected set. No rule assumes that every audit failure is repairable within the degree budget.

Preservation of the invariant. Initially every protected edge is a build edge. A learned replacement retains the entire active protected set and replaces an equal number of unprotected entries. Pin installation protects an edge already installed; the admission test ensures capacity at every recorded level. During promotion, the new prefix contains the old prefix, pins do not change, and restoration removes only edges outside $T(u) = P_{K_{\max}}(u) \cup \mathrm{pin}(u)$. The admission test gives $|T(u)| \le d_u$, where $d_u = |N^{(0)}_{\mathrm{out}}(u)|$. For any current list N of size d_u, $|N \setminus T(u)| = d_u - |N \cap T(u)| \ge |T(u) \setminus N|$. There are therefore enough removable entries to restore every missing required edge. Each intermediate list retains the old protected set, all installed recorded edges, and the original degree. Completion makes every new protected set present before activation. Demotion only shrinks protected sets. These cases establish the invariant by induction, assuming the atomic publication protocol in Appendix I. Sector-write recovery at fixed protection and layout is the separate claim of Lemma B.7.

Identifiers and quantized lengths use about $5K_{\max}$ bytes per node, or 160 B at $K_{\max} = 32$, before certification bits, flags, pins, and headers.

B.3 Covering and distance scales

Lemma B.1 (RobustPrune is α-covering). *Let $C \subseteq X \setminus \{u\}$ and let $S(u) = \textsc{RobustPrune}(u, C, \alpha, \infty)$ be the uncapped prune. Then for every $y \in C$, either $y \in S(u)$ or there exists $v \in S(u)$ with $\alpha \cdot \mathrm{dist}(v, y) \le \mathrm{dist}(u, y)$. In particular, taking $C = X \setminus \{u\}$ for every u yields an α-navigable graph.*

Proof. An uncapped run stops only when its working set is empty. Every candidate was either selected or removed by a selected point satisfying the pruning inequality. Selected points are never removed from that output. A selected target witnesses itself with distance zero. Taking the full collection as the candidate set therefore gives a witness for every target. □

The argument does not cover candidates left by a degree-capped invocation.

Lemma B.2 (Full covering size). *For a finite metric collection with doubling dimension d_0 and aspect ratio Δ_X, let $B_\alpha = 2^{d_0 \lceil \log_2(4\alpha) \rceil}$. The uncapped prune satisfies $|\textsc{RobustPrune}(u, X \setminus \{u\}, \alpha, \infty)| \le B_\alpha(1 + \lfloor \log_2 \Delta_X \rfloor)$.*

Proof. Set $r_{\min} = d_{\min}$, the smallest positive pairwise distance in X. Partition distances from u into annuli $[2^j r_{\min}, 2^{j+1} r_{\min})$. Two retained points in one annulus are separated by more than $2^j r_{\min}/\alpha$, since the later point survived the earlier point's pruning test. The annulus lies in a ball of radius $2^{j+1} r_{\min}$. After $\lceil \log_2(4\alpha) \rceil$ halvings, this ball is covered by at most B_α balls of radius at most $2^j r_{\min}/(2\alpha)$. Each contains at most one retained point. There are at most $1 + \lfloor \log_2 \Delta_X \rfloor$ nonempty annuli. □

Lemma B.3 (Supergraph monotonicity). *If S is α-navigable and $S \subseteq E(G)$, then G is α-navigable. Exact greedy search reaches any corpus target within $\lfloor \log_\alpha \Delta_X \rfloor + 1$ hops from any start.*

Proof. Every witness in $S(u)$ remains available in G. Exact greedy search chooses a neighbor at least as close to the target as that witness, so after m steps its distance is at most $D_X \alpha^{-m}$. For $m = \lfloor \log_\alpha \Delta_X \rfloor + 1$ this is less than $d_{\min}$, which is possible only at the target. The extra boundary step is needed, for example, when a two-point collection has aspect ratio one. □

Long edges are useful for this purpose because, under metric distances, making a large reduction in distance to a distant target requires a sufficiently long edge. Length alone, however, does not establish that an edge leads toward the target. The coverage check tests this additional requirement on the logged candidates.

The next lemma explains why an outer covering can be small. It groups retained candidates by length and bounds their number at each distance scale using the separation imposed by pruning. The statement concerns an uncapped run over the candidate log. It motivates a compact long-edge prefix, but does not certify the final degree-capped Vamana list. The coverage check in Section 4 tests that final prefix on the logged candidates.

Lemma B.4 (Outer coverings are thin). *Let $S(u)$ be the uncapped prune over a nonempty candidate log C_u in a metric of doubling dimension d_0, and let $D_u = \max_{p \in C_u} \mathrm{dist}(u, p)$. For $r > 0$, every logged candidate at distance greater than r has a kept witness longer than $t = (1 - 1/\alpha)r$. There are no kept edges longer than t if $D_u \le t$; otherwise their number is at most $B_\alpha(1 + \lfloor \log_2(D_u/t) \rfloor)$, with B_α as in Lemma B.2.*

Proof. A selected target witnesses itself. Otherwise its pruning witness v satisfies $\mathrm{dist}(v, y) \le \mathrm{dist}(u, y)/\alpha$. The triangle inequality gives $\mathrm{dist}(u, v) \ge (1 - 1/\alpha)\mathrm{dist}(u, y) > t$. If $D_u \le t$, no edge is longer than t. Otherwise partition $(t, D_u]$ into dyadic annuli and apply the

packing argument of Lemma B.2. At most $1 + \lfloor \log_2(D_u/t) \rfloor$ annuli are needed, with a separate boundary point included when D_u/t is a power of two. □

The next theorem gives the full navigability statement summarized in Section 4.3. Here, Δ_X is the ratio of the largest to the smallest nonzero pairwise distance.

Theorem B.5 (Scale-split navigability). *Assume metric distances, faithful logs, a fixed protection level, and invariant* (4)*. Exact greedy descent toward $y \in X$ makes α-progress at every certified or non-violating node above its landing radius. Any consecutive sequence of such progress steps has length at most $\lfloor \log_\alpha \Delta_X \rfloor + 1$ before reaching the target or leaving these conditions. If no node is violating, the graph is α-navigable. For exact greedy search, the probability of failing on a uniform corpus target is at most the probability of visiting a violating node. Uniform audits of the deployed search estimate its success event* (5) *without bias for a fixed graph, layout, and search/cache state.*

Proof of Theorem B.5. At a certified node above its landing radius, certification supplies a logged-range witness in the protected prefix. Faithfulness extends the coverage to the corpus target. At any other non-violating node, the current full out-list covers its log and hence the corpus (apply faithfulness with radius zero). Exact greedy search can take a progress step in either case. Lemma B.3 gives the bound on consecutive steps satisfying these conditions.

If no node violates, certified nodes cover the outer range by their prefix and the remaining range by non-violation; all other nodes cover their entire log. Thus faithfulness gives a witness for every non-target node. More generally, a failed exact greedy search must encounter a violating node, since otherwise its distances strictly contract until it reaches the target. Taking probabilities gives the failure bound. For a fixed graph, layout, and search/cache state, a uniform audit is an indicator of the deployed event in (5), so its expectation is exactly $\mathrm{Reach}(G)$, with layout and search/cache state implicit. The event bound for exact greedy search is not asserted for PQ beam search. □

Lemma B.6 (Progress above the landing radii). *Under the assumptions of Theorem B.5, consecutive exact greedy steps through certified or non-violating nodes above their landing radii satisfy $\mathrm{dist}(u_i, y) \le \mathrm{dist}(u_0, y)\alpha^{-i}$. At certified nodes the witness is in the protected prefix; at other non-violating nodes it is in the current full out-list. This establishes the progress property at those scales, not a particular route or its visitation probability.*

Proof. Apply Theorem B.5 at each eligible step and multiply the distance reductions. Different suffixes can choose different intermediate nodes even while this inequality holds. Thus preservation of progress does not establish the visitation floor in A1; that remains an assumption. □

Lemma B.7 (Protection under atomic suffix writes). *Fix S and the layout. Assume each sector is written atomically across power failure, each adjacency list fits within one sector, and both its old and new versions contain $S(u)$. Then every recovered mixture of these sector images satisfies* (4)*. No undo or redo of the suffix changes is needed to recover this invariant. The claim does not establish recovery of a layout swap or a protection increase whose restorations are incomplete.*

Proof. Each recovered sector is an old or new image containing the same protected set. Their mixture therefore satisfies (4), independently of disposable learner statistics. □

B.4 The restricted fixed point

Proposition B.8 (Restricted rewiring stabilizes). *Under A1–A3, fix the pools and protected set, retain state for every analyzed node, and use exact labels and uninterrupted running means. Suppose the empirical best set is installed at each batch without a gain gate. Then, after an almost-surely finite time with finite expectation, every analyzed node selects its oracle top-$R_\ell(u)$ eligible candidates and the graph is constant. The same conclusion applies to fixed cost-adjusted scores when A2–A3 hold for those scores.*

Proof. Fixed finite pools give finitely many possible out-lists. A3 and the boundary gap imply that the oracle set is the same across reachable graphs: changing each of two scores by at most η cannot reverse a boundary difference greater than 2η.

For a candidate, its conditional reward means lie in an interval of width at most η. The running mean differs from the average of those conditional means by at most ϵ except with probability $2e^{-2m\epsilon^2}$ after m observations, by the bounded-difference martingale inequality. Thus its error relative to any current mean is at most $\eta + \epsilon$. Choose $\epsilon < (\Delta^\flat - 2\eta)/2$. A union bound over the finite set of candidates and a summation over m show that boundary errors eventually stop almost surely. A1 supplies observations with conditional probability at least $\mu_{\min}$ per query; the same concentration argument for visit counts gives an exponentially decreasing query-time tail and finite expected stabilization time. The next batch installs the common oracle sets, after which the graph is unchanged. □

C COST ADJUSTMENT

Proof of Theorem 6.1. Fix reference measurements μ_0, Q_0, c_0. Define

$$\bar{F}(G) = \sum_u \mu_0(u) \sum_{v \in N^G_{\mathrm{out}}(u)} Q_0(u, v),$$
$$\bar{I}(G) = \sum_u \mu_0(u) \sum_{v \in N^G_{\mathrm{out}}(u)} c_0(u, v).$$

Nonnegative node weights make the weighted objective separable over out-lists. The prescribed local selection maximizes it. For $\theta > 0$, a graph with no smaller utility and no larger cost, one strict, would have a larger weighted objective. This proves Pareto optimality. At zero weight only utility optimality follows.

For $\theta' > \theta$, optimality at each weight gives

$$\bar{F}(G) - \theta\bar{I}(G) \ge \bar{F}(G') - \theta\bar{I}(G'),$$
$$\bar{F}(G') - \theta'\bar{I}(G') \ge \bar{F}(G) - \theta'\bar{I}(G).$$

Adding proves $(\theta' - \theta)(\bar{I}(G) - \bar{I}(G')) \ge 0$.

For the controller, a positive limiting weight makes the projection inactive eventually. With $\gamma_t = a/t$, $\widehat{I}_t - I^\star = t(\theta_{t+1} - \theta_t)/a$.

Summation by parts yields, up to finitely many initial terms,

$$\frac{1}{T}\sum_{t=1}^{T}(\widehat{I}_t - I^\star) = \frac{1}{a}\left(\theta_{T+1} - \frac{1}{T}\sum_{t=1}^{T}\theta_t\right) + o(1).$$

Both terms in parentheses converge to the same limit. □

Controller convergence needs additional conditions; an inactive constraint can leave I/O below the target.

D SELECTION AND WRITE BOUNDS

Theorem D.1 (Hysteresis below the gap). *Use the fixed-pool, exact-label running-mean setting of Proposition B.8, with fixed protection, per-node quotas, costs, and weight θ. Require A2–A3 for the adjusted utilities $Q^\star_G(u,v) - \theta c(u,v)$ on $\mathcal{E}(u)$, with uniform boundary gap $\Delta^\flat_\theta > 0$ and adjusted-score drift below $\Delta^\flat_\theta/2$. If $0 \le \lambda < \Delta^\flat_\theta$, gated selection converges almost surely to the same oracle set as ungated selection and eventually issues no further commits. For any $\lambda \ge 0$, if the current score error is at most e, an incumbent that does not pass the gate has local deficit at most $\lambda + 2R_\ell e$. Under a fixed reference visitation law, the corresponding per-query deficit is at most $(\lambda + 2R_\ell e)\bar{v}$.*

Proof. By the adjusted-score version of A3 and the eligible boundary gap, the oracle set is common across reachable graphs. Each inside–outside cost-adjusted difference has conditional mean at least $\Delta^\flat_\theta$. The paired running observations concentrate around their average conditional means. Eventually every such empirical difference exceeds any fixed $\lambda < \Delta^\flat_\theta$. An incorrect incumbent omits an inside candidate and includes an outside candidate; the empirical best set therefore improves it by more than λ. The next batch installs the common oracle set. It then proposes the same set and has zero gain.

For the finite-error statement, the difference of two set sums has error at most $2R_\ell e$. The empirical best set has gain at least the gain of the true best set under the same estimated scores. If its gain is at most λ, the true deficit is at most $\lambda + 2R_\ell e$. Weighting by reference visitation and using $\sum_u \mu(u) = \bar{v}$ gives the per-query bound. This finite-error statement does not require convergence to an absorbing graph when the threshold exceeds the gap. □

We next bound the number of accepted replacements. Consider T served queries, indexed by t, and n_+ analyzed nodes with positive visitation probability. Each node has at most C candidates and at most R_ℓ replaceable out-list entries. Let N_T count commits that pass the threshold $\lambda > 0$, before the write queue combines them.

At query t, candidate v at node u has utility estimate $\widehat{Q}_t(u,v)$, read-cost estimate $c_t(u,v)$, and adjusted score $s_t(u,v) = \widehat{Q}_t(u,v) - \theta_t c_t(u,v)$. The cost weight θ_t has a fixed finite upper bound $\theta_{\max}$. Its step size γ_t is zero between batch updates. For the tracking bound, β bounds the combined change in an adjusted score per query, including changes in utility, cost, and the cost weight. Bounding the EWMA step alone does not establish this bound. The sum below ranges over the analyzed nodes, their pool candidates, and consecutive query steps.

Theorem D.2 (Write complexity). *On an interval with fixed pools and protection and $\lambda > 0$, let N_T count gated out-list commits and let $s_t(u,v) = \widehat{Q}_t(u,v) - \theta_t c_t(u,v)$, with $0 \le \theta_t \le \theta_{\max}$ and estimates and costs in $[0,1]$. Then*

$$N_T\lambda \le n_+R_\ell(1+\theta_{\max}) + \sum_{u,v}\sum_{t<T}|s_{t+1}(u,v) - s_t(u,v)|.$$

Running means for both utilities and costs, and controller steps $\gamma_t = O(1/t)$ with bounded I/O observations, give $N_T = O(n_+C\log(T+1)/\lambda)$. Updates bounded by a combined rate β give $N_T = O((n_+R_\ell + n_+C\beta T)/\lambda)$. After a protection change, pool replacement, estimator reset, or forced restoration, apply the bound to the resulting intervals and add their potential ranges and forced writes. Under the exact-selection conditions of Theorem D.1, fixed-level commits eventually stop almost surely. The tracking bound is an upper bound, not a universal lower bound.

The theorem explains why more queries need not cause proportionally more graph writes. Every accepted replacement must improve the estimated adjusted score by more than λ. With fixed scores, only a finite amount of improvement is available. New observations can change scores and create further opportunities. The first term in the bound accounts for the initial room for improvement; the second accounts for the total subsequent score change.

Proof of Theorem D.2. For current learned sets $A_t(u)$, use the potential $\Phi_t = \sum_u \sum_{v\in A_t(u)} s_t(u,v)$. Its range has width at most $n_+R_\ell(1+\theta_{\max})$. Each gated commit raises it by more than λ. At fixed out-lists, score updates decrease it by at most their total absolute variation. Telescoping proves the first inequality.

For $s_t = \widehat{Q}_t - \theta_t c_t$, total variation is bounded by

$$\mathrm{TV}(s) \le \mathrm{TV}(\widehat{Q}) + \theta_{\max}\mathrm{TV}(c) + n_+C\mathrm{TV}(\theta).$$

A running mean changes by at most $1/m$ at its m-th sample, giving $O(\log(T+1))$ variation per candidate. A bounded I/O error and $\gamma_t = O(1/t)$ give the same harmonic bound for the controller. Updates of magnitude $O(\beta)$ give linear variation instead. Splitting at changes of pools, protection, or estimator state gives one range term per segment. Forced restorations do not pass the gain gate and must be counted separately. Coalescing commits can reduce physical writes; metadata and layout writes are additional work. □

Diminishing steps alone need not give logarithmic variation: consider $\gamma_t = t^{-3/4}$. An EWMA can also see changing rewards with no ranking changes, so no universal positive write-rate lower bound follows.

Proof of Theorem 6.2. Divide the first inequality in Theorem D.2 by λ. Its range term bounds the initial room for improvement, and its variation term accounts for subsequent score changes. The logarithmic conclusion follows from the running-mean and controller bound in that theorem, on intervals without resets or forced restorations. □

E LEARNER MEMORY

Proposition E.1 (Memory–quality tradeoff). *Consider i.i.d. reference queries with a fixed visitation and reward law, fixed pools and feasible per-node quotas, exact labels, a gap Δ, and ungated running-mean selection on a fixed set of the H hottest nodes. Against*

the oracle choices on the same visits, expected cumulative local utility loss is

$$O(HR_\ell C/\Delta^2 + HR_\ell B) + R_\ell \bar{v}\varepsilon_H T,$$

where ε_H is the fraction of expansions outside the set and B is the fixed batch period. If $\mu(i) \le ci^{-s}$ with $s > 1$, and the other parameters are fixed, choosing $H = \Theta((\Delta^2 T)^{1/s})$ gives $O(T^{1/s})$ loss, subject to $H \le n$. Admission errors, resets, approximate labels, and a nonzero gate require their own terms; the result does not assert that aging LFU equals the oracle hot set.

Proof. After m visits to a cached node, the probability that its empirical ranking crosses a gap Δ is at most $2C\exp(-m\Delta^2/2)$, by bounding all score errors by $\Delta/2$. Summing over $m \ge 0$ gives $O(C/\Delta^2)$ expected erroneous visit decisions. Each loses at most R_ℓ utility. Under the fixed i.i.d. law, a node's observed reward sequence is independent of its inter-visit waiting times. A batch delay contributes an additional $O(R_\ell B)$ visits per node: at a batch boundary the latest observation count lags by at most B, so summing the same exponential bound with this lag adds at most B initial terms.

An uncached node loses at most R_ℓ on a visit. Its expected contribution is therefore at most $R_\ell T\mu(u)$. The sum outside the hot set is $\bar{v}\varepsilon_H$, giving the tail term. Under the Zipf upper envelope, $\sum_{i>H}\mu(i) \le cH^{1-s}/(s-1)$. Balancing a term proportional to H/Δ^2 with TH^{1-s} gives the stated expected-loss scaling. □

Proposition E.2 (Sample requirement in the cold tail). *For Bernoulli candidate rewards with means bounded away from zero and one, there are pairs separated by Δ for which any rule based only on their observations needs $\Omega(1/\Delta^2)$ samples to identify the better candidate with probability at least $2/3$ under both possible orderings. With T queries and visitation probability $\mu(u)$, this requires $\mu(u)T = \Omega(1/\Delta^2)$ in the worst case. Under $\mu(i) \le ci^{-s}$, at most $O((c\Delta^2 T)^{1/s})$ nodes meet this sampling-rate threshold. This is not a lower bound for every reward pair or for methods with useful side information.*

Proof. Compare two instances with observation $(Z, 1-Z)$, swapping the Bernoulli means $1/2 + \Delta/2$ and $1/2 - \Delta/2$, for sufficiently small Δ. Their per-observation KL divergence is $O(\Delta^2)$. A test with error at most $1/3$ under both instances needs a constant total information distance, hence $\Omega(1/\Delta^2)$ observations. In T queries the expected information is $O(\mu(u)T\Delta^2)$, which gives the necessary visitation scale for these hard instances. The Zipf envelope limits the number of nodes above that scale. At means near the endpoints, or with informative geometry or shared observations, this lower bound need not describe the task. □

F BLOCK LAYOUT

Exact optimization. The general fixed-trace problem is NP-hard already at $\kappa = 3$. Reduce from partitioning a graph into triangles [14]. For a graph with n vertices and $m > 0$ edges, assign probability $1/m$ to the two-vertex trace of each edge. A layout saves the number of graph edges placed within blocks, divided by m. Capacity three permits at most n such edges, with equality exactly when all vertices are partitioned into triangles. Thus savings of n/m decide triangle partition.

Fix the graph and an uncached visitation law. For a block b, let $h(b) = \mathbb{P}[\mathcal{V}(q) \cap b \ne \emptyset]$, $g(b) = \sum_{u\in b}\mu(u) - h(b)$, and $W_b = \sum_{\{u,v\}\subseteq b}\mathrm{co}(u,v)$. Savings relative to separate reads are $\mathrm{Sav}(\pi) = \sum_b g(b)$. The pairwise objective is $W(\pi) = \sum_b W_b$. Trees on singleton blocks have weight zero, and $\kappa \ge 2$ in the approximation statement.

Lemma F.1 (Savings sandwich). *For any b with $|b| \le \kappa$, with $\mathrm{MST}(b)$ the maximum-weight spanning tree of b under weights co: $\frac{2}{\kappa}W_b \le \mathrm{MST}(b) \le g(b) \le W_b$.*

Proof. For a realized trace, let $Z = |\mathcal{V}(q) \cap b|$. Savings in the block are $(Z-1)_+$ and the number of visited pairs is $\binom{Z}{2}$, so $g(b) \le W_b$ after taking expectations. The visited vertices of any tree induce at most $(Z-1)_+$ tree edges; hence the expected weight of every tree is at most $g(b)$. Finally, the stars centered at the block's vertices count each pair twice. Their average weight is $2W_b/|b|$, so the maximum-weight spanning tree has at least $2W_b/\kappa$ weight. This also gives a direct proof of the union-bound relation used in Hunter's inequality [9]. □

Theorem F.2 (Pairwise packing approximates I/O savings). *Fix the graph and visitation law, and let blocks hold κ nodes. Any ρ-approximate maximizer of within-block co-visitation weight achieves at least $\frac{2\rho}{\kappa}$ of the optimal expected-I/O savings. An exact pairwise maximizer is a $\kappa/2$-approximation, and at $\kappa = 2$ the reduction is exact: the optimal layout is a maximum-weight matching, computable in polynomial time.*

Proof. Let $\pi^\star$ maximize savings and let π_W satisfy $W(\pi_W) \ge \rho\max_\pi W(\pi)$. Then

$$\begin{aligned}\mathrm{Sav}(\pi_W) &\ge (2/\kappa)W(\pi_W)\\ &\ge (2\rho/\kappa)W(\pi^\star)\\ &\ge (2\rho/\kappa)\mathrm{Sav}(\pi^\star).\end{aligned}$$

For pairs, inclusion–exclusion gives $g(\{u,v\}) = \mathrm{co}(u,v)$, while a singleton has zero savings. Thus the exact problem at capacity two is maximum-weight matching, allowing unmatched nodes as singletons. □

Proposition F.3 (What improving layout swaps establish). *With pairwise weights fixed for an entire sequence, strictly improving swaps terminate after finitely many steps. If every positive-gain swap in a specified neighborhood is eligible and the procedure stops only when none remains, the final layout is a local optimum for that neighborhood. With a gain threshold $\lambda_\pi \ge 0$, exhausting the eligible swaps at the final layout guarantees only that none has gain above λ_π. Both topology changes respecting* (4) *and layout changes preserve the protected edge set. Increasing pairwise weight does not, in general, increase true I/O savings, and these facts do not establish convergence of the alternating online loop under changing estimates.*

Proof. There are finitely many layouts, and strict increases of a fixed W cannot revisit one. Under the stated stopping rule, an improving swap at the final layout would contradict termination. With the threshold, the same argument excludes only eligible swaps with gain above λ_π; smaller positive gains can remain. Protected-set preservation follows from the edge-selection constraint and from layout changes leaving adjacency intact.

For blocks $\{0,1,2\}, \{3,4,5\}$ and traces $\{0,2\}, \{0,1,3\}$ with probabilities 0.6 and 0.4, swapping 2 and 3 raises W from 1 to 1.2 but

lowers savings from 1 to 0.8. Thus pairwise improvement need not improve I/O, and changing objectives need not prevent cycles on a finite layout set. □

G APPROXIMATE LABELS

Let $A_{\mathrm{pr}}(q)$ be the probe result and $g_{\mathrm{pr}} = \mathbb{E}[1 - |A_{\mathrm{pr}}(q) \cap N_k(q)|/k]$. Both sets have size k. Let $Q^{\star}_{\mathrm{pr}}$ replace the exact answer in (2) by the probe result.

Lemma G.1 (Per-edge probe bias). *For every $u \in V_+$ and every v,*

$$\left|Q^{\star}_{\mathrm{pr}}(u,v) - Q^{\star}(u,v)\right| \;\le\; \mathbb{P}\left[v \in A_{\mathrm{pr}}(q) \triangle N_k(q) \;\middle|\; u \in \mathcal{V}(q)\right] \;\le\; \frac{2k\, g_{\mathrm{pr}}}{\mu(u)}.$$

Proof. Set $D(q) = A_{\mathrm{pr}}(q) \triangle N_k(q)$. Membership indicators differ only on $D(q)$, proving the first inequality after conditioning. Also $\mathbb{E}|D(q)| = 2kg_{\mathrm{pr}}$ and $\mathbb{P}[v \in D(q)] \le \mathbb{E}|D(q)|$. Conditioning on a visit to u divides this bound by at most $\mu(u)$. □

Proposition G.2 (Labels that preserve the selected set). *Fix a reference graph, pools, protection, quotas, costs, and θ; use uninterrupted running means at nodes with positive visitation. Let $b_{\mathrm{pr},u}$ and $b_{\mathrm{self},u}$ bound the per-edge utility bias of probes and self-labels. For independent probe selection at rate p, the mixed mean used by Algorithm 1 has bias at most $b_u = (1-p)b_{\mathrm{self},u} + pb_{\mathrm{pr},u}$. Let $\Delta^{\flat}_{\theta} > 0$ be the uniform true adjusted gap at the eligible selection boundaries. If $2b_u < \Delta^{\flat}_{\theta}$ at each analyzed node, its estimated top-$R_\ell(u)$ set eventually equals the oracle set almost surely. No claim is made about ordering within the selected or excluded group. For a current sampling error e_u, an incorrect incumbent passes the gain gate whenever $\lambda < \Delta^{\flat}_{\theta} - 2R_\ell(b_u + e_u)$. For probe-only observations, one may instead use $b_u \le 2kg_{\mathrm{pr}}/\mu(u)$; obtaining a fixed number of those observations takes $1/p$ times as many visits in expectation.*

Proof. Independent label-source selection makes the conditional mixed mean $(1-p)Q_{\mathrm{self}} + pQ_{\mathrm{pr}}$. The triangle inequality gives its bias bound. An inside–outside score difference changes by at most $2b_u$, so a smaller bias preserves the boundary ordering. Running means converge to the mixed means, giving eventual equality of selected sets. Including sampling error and comparing two sets of at most R_ℓ entries gives the gain condition by Theorem D.1. Independent probe selection takes $1/p$ visits per probe observation in expectation. □

For true utilities $(0.6, 0.4)$, self-label utilities $(0.1, 0.9)$, perfect probes, and $p = 0.01$, the mixture $(0.105, 0.895)$ reverses the order. Positive probe probability therefore does not remove self-label bias. Fixed-pool analysis also separates admission-time observations from later samples.

H AUDITS AND THE GOVERNOR

Reference protection level. For fixed θ, let $\widehat{G}_{P_j,\theta}$ be the value-greedy fixed point at level j (Section 6.2). If these graphs exist and the target is feasible, define $k^{\star}_{\mathrm{p}} = \min\{j : \mathrm{Reach}(\widehat{G}_{P_j,\theta}) \ge 1 - \varepsilon_{\mathrm{r}}\}$. The governor does not compute these reference graphs or guarantee convergence to this analytical target (Proposition H.2).

The governor's confidence sequence uses only uniform-target audits to certify (5); recent-insert outcomes are reported separately. The parameter p_{u} controls how often uniform targets are sampled; the confidence bound depends on their observed count, not on a particular choice of p_{u}. A window starts at initialization, a completed level change, or a harm response; each start depends only on observations already available. Checks use prefixes of that window, not selected subsets of past audits. Number windows and resets by $j \ge 1$ and allocate $\delta_j = \delta/[j(j+1)]$. For the first m uniform audits in window j, let Z_i be the miss indicator and $p_i = \mathbb{E}[Z_i \mid \mathcal{F}_{i-1}]$. Layout and search state are implicit arguments of $\mathrm{Reach}(G)$. The graph and search policy must be fixed for each audit before drawing its target, giving $p_i = 1 - \mathrm{Reach}(G_i)$. The publication latch in Appendix I prevents an audit from spanning graph or layout changes. Its acquisition precedes target selection, so synchronization does not select targets by difficulty.

Let $\widehat{p}_{j,m} = m^{-1}\sum_i Z_i$ and use

$$a_{j,m} = \sqrt{\frac{\log(2m(m+1)/\delta_j)}{2m}},$$

with $\mathrm{LCB} = \max(0, \widehat{p}_{j,m} - a_{j,m})$ and $\mathrm{UCB} = \min(1, \widehat{p}_{j,m} + a_{j,m})$. The bound concerns the average conditional miss probability in the window. Each completed audit contributes one outcome, tagged by sampling stratum; per-node diagnostic records do not count as additional Bernoulli trials.

Theorem H.1 (Anytime audit certificate). *Use the uniform-target audit substream and the confidence construction in Appendix H, with total error budget δ across all windows and resets. With probability at least $1-\delta$, every current demotion certificate bounds the average reachability at its audit instants by at least $1 - \varepsilon_{\mathrm{r}} + \varsigma$. For a later time t,*

$$\mathrm{Reach}(G_t) \ge 1 - \varepsilon_{\mathrm{r}} + \varsigma - D_t,$$

where D_t sums the positive decreases in reachability caused by all graph, layout, and search-state changes since the earliest audit. This result does not bound D_t by the number of edits or by their utility gains.

Proof of Theorem H.1. The centered outcomes $Z_i - p_i$ are bounded martingale differences. At a fixed m, the probability that their average exceeds $a_{j,m}$ in absolute value is at most

$$2e^{-2ma^2_{j,m}} = \frac{\delta_j}{m(m+1)}.$$

Summing over m and j gives total failure probability at most δ. Whenever the demotion test passes, the window's average miss probability is therefore at most $\varepsilon_{\mathrm{r}} - \varsigma$ on this common event.

For an audit instant s, telescoping all subsequent changes yields $\mathrm{Reach}(G_t) \ge \mathrm{Reach}(G_s) - D_{s,t}$, where $D_{s,t}$ sums their positive reachability decreases. The sum D_t starting at the earliest audit is at least each $D_{s,t}$. Average the inequalities over the audit instants. Changes include suffix rewires, reserve claims, restorations, layout, and other search-state changes affecting the success event. Restoring build edges does not necessarily undo every earlier suffix or layout change. □

Proposition H.2 (Limits of protection-level convergence). *Monotone fixed-level reachability and arbitrarily long settling intervals do not by themselves imply finitely many transitions of Algorithm 3. A safe level can repeatedly certify a demotion to an unsafe level, followed by promotion. Monotonicity alone also gives no one-level bound between the smallest level satisfying the reachability target and the smallest level satisfying a stricter certificate with fixed slack ς.*

Proof. Consider levels zero and one. Let their settled miss rates be respectively above ε_r and below $\varepsilon_r - \varsigma$, with the other demotion gates passing at level one. Even with exact rates, the governor demotes from one to zero and then promotes back to one. After the dwell it can do so again, because its rule does not record that this same lower level failed. These rates are monotone and decisions can wait arbitrarily long for settling.

Arbitrarily many consecutive levels can have miss rates in $(\varepsilon_r - \varsigma, \varepsilon_r]$ before one falls below $\varepsilon_r - \varsigma$. All plateau levels meet the target but fail the stricter certificate. Monotonicity places no bound on this gap in levels. □

Proposition H.3 (Adaptation cost of the tier). *At audit rate p_a, expected audit search I/O generated per served query is $p_a \mathbb{E}[I_{audit}]$, plus target loading and diagnosis. The governor uses $O(K_{max})$ global counters, bounded buffers, and bounded metadata per record. Starting from a build with all recorded edges installed, fix the corpus and recorded build lists. A* reserve-edge claim *is the removal of one installed recorded build edge by a learned replacement. Under the restoration rule in Appendix B.2, reserve-edge restorations are at most the number of such claims over the same execution prefix. Physical sector writes, restoration scans, metadata, layout, and other repair costs are accounted for separately.*

Proof. Linearity of expectation gives the audit cost. The rank counters use $O(K_{max})$ entries. Initially all recorded edges are installed. Restoration never removes a recorded edge, and demotion removes no edge. With corpus and build lists fixed, each missing recorded edge was therefore removed by a learned replacement. Charge its restoration to that removal. Restoration inserts only missing edges, so another restoration of the same edge requires a new claim. This gives an injection and proves the event-count bound. Several claims can share a sector write while their restorations occur in separate writes, so the injection does not bound the ratio of physical writes. Forced restorations also require separate accounting in Theorem D.2 because they need not pass its gain gate. □

The mean audit I/O in Proposition H.3 averages the costs of uniform and recent-insert targets with weights p_u and $1 - p_u$ when the recent-insert stratum is nonempty. If it is empty, all audits are uniform. With a nonempty stratum, independent sampling, and fixed probabilities $p_a > 0$ and $p_u > 0$, the nominal sampling requirement for m uniform audits is $m/(p_a p_u)$ served queries in expectation. This is the negative-binomial mean when each selected audit is processed without delay. It excludes worker queueing, synchronization, and execution delays; completed audits determine when a confidence check is available.

On a fixed index, a single check after m predeclared uniform audits with zero failures gives the one-sided miss-rate bound $1-\delta^{1/m} \le \log(1/\delta)/m$. At $\delta = 0.05$, about 1,500 audits suffice for $\varepsilon_r = 0.002$. For example, with $p_a = 0.01$ and $p_u = 0.9$, the nominal sampling requirement is about 1.7×10^5 served queries in expectation. The stricter demotion threshold $\varepsilon_r - \varsigma = 0.0015$ requires about 2,000 failure-free audits. These single-check counts do not apply to repeated checks and resets. For the actual anytime formula, at $\delta = 0.05$ in the first window and with zero misses, thresholds 0.002 and 0.0015 require respectively 4,370,342 and 8,040,449 uniform audits. The latter has a nominal sampling requirement of about 8.93×10^8 served queries at $p_a = 0.01$, $p_u = 0.9$, before worker delays. This conservative sample requirement can substantially delay demotion; the fixed-check example is not its operating sample budget.

Diagnostic statistics. The same window collects observed pairs (u, y), with u expanded by search and y its query target. For rank j, count pairs for which the build edge v_j exists and remains in the list. Use the PQ progress test of Algorithm 2. The fraction with another retained neighbor that passes the test and has PQ target distance no greater than that of v_j is $\mathrm{red}[j]$. The fraction for which v_j passes and no other retained neighbor passes is $\mathrm{sw}[j]$. With no observations, set these to zero and one, respectively, to prevent demotion. These are empirical diagnostics, not confidence bounds on global coverage. Repair volume is the number of node–target diagnostics proposing a repair divided by the number of served queries in the window; it is zero at initialization. χ_{rep} uses this same unit. The diagnosed rank j^+ is the largest reserve rank among outstanding diagnoses; absent such a rank, harm requests the next level, capped at K_{max}. A drift alarm is an optional operator-supplied signal, disabled unless configured; no theorem assumes a drift detector. Recent inserts are records added within an operator-set recent-query window; sampling uses a disk-resident insert index with bounded RAM. The wait T_{dwell} and the diagnostic thresholds are operating parameters; they do not imply convergence (Proposition H.2).

I IMPLEMENTATION DETAILS

Records and statistics. Restoration identifiers and quantized lengths use $5K_{max}$ bytes per node, before headers, certification bits, and pins. The full record must fit the configured layout. At $C = 128$, identifiers and two 16-bit fields alone use 1,024 bytes; additional counts, buffers, and sketches must be budgeted. Running means require counters that do not saturate or lose updates. New candidates and admitted nodes start fresh estimates; cost samples record availability at the source expansion, as defined in (6). The fixed-pool proofs exclude admission/reset intervals unless their costs are added separately. Proposal buffers retain at most C entries per cached node, deduplicate identifiers, and drop oldest proposals on overflow. Proposal helpers discard requests for flagged nodes. The candidate pool always retains the current out-list before admitting proposals, so equal-cardinality replacement is feasible.

Publication and recovery. A block is the unit counted in (1); a sector is the storage stack's atomic write unit. A block or record spanning several sectors therefore requires the transaction protocol below. Serving searches hold a shared publication latch for their index reads. An audit acquires it before drawing its target

and holds it through recording the outcome. Background writers acquire it exclusively when publishing an adjacency, pin, level, or layout change. Availability within an audit includes its own fetched blocks; its initial resident-cache view is fixed. This specification provides consistent observations; its synchronization cost is part of maintenance overhead.

Accepted graph changes use a bounded queue with backpressure and at most one pending replacement per node. Before accepting a change, validate the node version and its gain against the current logical list; stale proposals are recomputed. No later graph/layout transaction is interleaved with a batch's publication. Queued writes are drained before a governor transition. For the learning analysis, this publication is the batch boundary: the next observations use the published graph. Physical writes can coalesce multiple record changes without changing their logical commit count.

For a multi-sector change, pin installation, or layout swap, a bounded write-ahead journal records the old and new records and mapping entries, and is flushed before in-place writes. Hold the exclusive latch, write and flush all destination sectors, durably mark the transaction committed, then expose the new mapping/state and release the latch. The journal's commit record fits in an atomic sector. Before serving after a crash, redo committed transactions and undo incomplete ones. The journal is checkpointed before reuse; the relocation map resides on disk with a bounded cache. This protocol covers multi-record publication; Lemma B.7 remains the narrower fixed-layout sector result.

A promotion persists its request before scanning, preserves the old active protected sets throughout, and publishes its new level only after all restorations are durable (Appendix B.2). Recovery resumes a pending scan before adaptation. Queue and journal capacities are operator budgets; reaching capacity delays maintenance rather than allocating unbounded state. These protocols specify required behavior; the mathematical results do not establish their latency or throughput.

Accounting for background reads. The probe and audit rates control how often background searches run. Their read costs also depend on search effort and the targets they reach. The expected additional reads per served query are the probe rate times the mean reads per probe, plus the audit rate times the mean reads per audit. A corpus-target audit can cost more than a typical workload query, especially when the workload benefits from a warm block cache. Thus a 1% audit rate need not mean 1% additional reads. The evaluation records these reads alongside serving I/O and reports maintenance writes separately.

Operating modes. Adapt runs the complete loop. *Freeze* stops learning and releases the hot-node cache while retaining the adapted index. *Rebuild* re-extracts protection records after a substantial corpus change. The design targets compatibility with stock DiskANN serving. Corpus insertion and deletion are handled by the underlying update mechanism, such as FreshDiskANN [21]; newly inserted records receive their length order and restoration information. The interaction with updates and the residual cost of the frozen index belong in the deployment evaluation.

REFERENCES

[1] Benjamin Arai, Gautam Das, Dimitrios Gunopulos, Vagelis Hristidis, and Nick Koudas. 2010. An Access Cost-Aware Approach for Object Retrieval over Multiple Sources. *Proc. VLDB Endow.* 3, 1 (2010), 1125–1136. https://doi.org/10.14778/1920841.1920982

[2] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. 2002. Finite-time Analysis of the Multiarmed Bandit Problem. *Machine Learning* 47, 2–3 (2002), 235–256.

[3] Nicolas Bruno and Surajit Chaudhuri. 2007. An Online Approach to Physical Design Tuning. In *Proc. IEEE Int. Conf. Data Engineering (ICDE)*. 826–835. https://doi.org/10.1109/ICDE.2007.367928

[4] Surajit Chaudhuri and Vivek R. Narasayya. 1997. An Efficient, Cost-Driven Index Selection Tool for Microsoft SQL Server. In *Proc. Int. Conf. Very Large Data Bases (VLDB)*. 146–155.

[5] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 17, 11 (2024), 2735–2749.

[6] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[7] Felix Halim, Stratos Idreos, Panagiotis Karras, and Roland H. C. Yap. 2012. Stochastic Database Cracking: Towards Robust Adaptive Indexing in Main-Memory Column-Stores. *Proc. VLDB Endow.* 5, 6 (2012), 502–513. https://doi.org/10.14778/2168651.2168652

[8] Steven R. Howard, Aaditya Ramdas, Jon McAuliffe, and Jasjeet Sekhon. 2021. Time-Uniform, Nonparametric, Nonasymptotic Confidence Sequences. *The Annals of Statistics* 49, 2 (2021), 1055–1080. https://doi.org/10.1214/20-AOS1991

[9] David Hunter. 1976. An Upper Bound for the Probability of a Union. *Journal of Applied Probability* 13, 3 (1976), 597–603.

[10] Stratos Idreos, Martin L. Kersten, and Stefan Manegold. 2007. Database Cracking. In *Proc. Biennial Conf. Innovative Data Systems Research (CIDR)*. 68–78.

[11] Piotr Indyk and Haike Xu. 2023. Worst-case Performance of Popular Approximate Nearest Neighbor Search Implementations: Guarantees and Limitations. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[12] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. 2022. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. *arXiv preprint arXiv:2211.12850* (2022).

[13] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[14] David G. Kirkpatrick and Pavol Hell. 1983. On the Complexity of General Graph Factor Problems. *SIAM J. Comput.* 12, 3 (1983), 601–609. https://doi.org/10.1137/0212040

[15] Robert Krauthgamer and James R. Lee. 2004. Navigating Nets: Simple Algorithms for Proximity Search. *Proc. ACM-SIAM Symp. Discrete Algorithms (SODA)* (2004), 798–807.

[16] Tor Lattimore and Csaba Szepesvári. 2020. *Bandit Algorithms.* Cambridge University Press.

[17] Liang Li, Shufeng Gong, Yanan Yang, Yiduo Wang, and Jie Wu. 2026. I/O Optimizations for Graph-Based Disk-Resident Approximate Nearest Neighbor Search: A Design Space Exploration. *Proceedings of the VLDB Endowment* 19, 7 (2026), 1484–1498. https://doi.org/10.14778/3801059.3801064

[18] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[19] Chandan K. Reddy, Lluís Màrquez, Fran Valero, Nikhil Rao, Hugo Zaragoza, Sambaran Bandyopadhyay, Arnab Biswas, Anlu Xing, and Karthik Subbian. 2022. Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search. arXiv:2206.06588 [cs.IR] https://arxiv.org/abs/2206.06588

[20] Herbert Robbins and Sutton Monro. 1951. A Stochastic Approximation Method. *The Annals of Mathematical Statistics* 22, 3 (1951), 400–407.

[21] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *arXiv preprint arXiv:2105.09613* (2021).

[22] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[23] Sebastiano Vigna. 2013. Quasi-Succinct Indices. In *Proceedings of the Sixth ACM International Conference on Web Search and Data Mining*. 83–92. https://doi.org/10.1145/2433396.2433409

[24] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. In *Proc. ACM Manag. Data (SIGMOD)*.

[25] Ziyang Yue, Bolong Zheng, Ling Xu, Kanru Xu, Shuhao Zhang, Yajuan Du, Yunjun Gao, Xiaofang Zhou, and Christian S. Jensen. 2025. Select Edges Wisely: Monotonic Path Aware Graph Layout Optimization for Disk-based ANN Search.

Proc. VLDB Endow. 18, 11 (2025), 4337–4349. https://doi.org/10.14778/3749646. 3749697